B.TECH CSE • OSDBMS-V-2026-T007 • GRAPHIC ERA DEEMED TO BE UNIVERSITY • TEAM CODEX4

OS-DBX: A Unified System for Tracking Computer & Database Problems

Complete Phase-II Project Presentation & Viva-Voce Q&A Master Reference Guide (Codebase, 32-Table Schema, Git History, Architecture & Live Testing)

1. Krishna Anasane (Lead)
ID: 24021788 | 42 Git Commits
DBMS Core, 32-Table Schema, Identity Bridge, Correlation, Replay, Auth & CLI

2. Akanksha Balonie
ID: 24022979 | DevOps & API
FastAPI Backend Integration, REST API Testing, GitHub CI & Diagnostics

3. Aditya Kumar
ID: 24011059 | 14 Git Commits
OS Process/Resource/FS Collectors, WindowsAdapter, Live Stream & OS Sim Lab

4. Akshita Negi
ID: 24022885 | 5 Git Commits
Web Frontend Playground (HTML/CSS/JS), Process Visualizer & Dashboard UI

Master Index of All 15 Viva Topics Covered in This Document:

Q1. Project Vision, Problem Statement, OS+DBMS Justification, USP, Correlation Engine & Identity Bridge

Q2. Why Python, Complete Full-Stack Tech Stack, Server IP/Port (`127.0.0.1:8000`) & Selection Rationale

Q3. JWT Credentials, Login Significance, Linux UID Mapping & Register New User Flow (Code + Schema)

Q4. OS-DBX Live Health & Device Monitor (Latency, Traces, TXs, Overhead), Heart of Project & Alternatives

Q5. Unified Cross-Layer Observability Console, Active Socket Map & Ad-Hoc SQL Causality Proof

Q6. OS Monitoring & Live Observatory: Host Telemetry, Pros/Cons, Process Table & Step-by-Step Testing

Q7. DBMS Monitoring & Live Query Observatory: MySQL Performance Schema Collector, Codebase & Concepts

Q8. Real-Time Cross-Layer Correlation Engine & Identity Bridge Map: Socket/PID Logic & OS/DBMS/CN Concepts

Q9. Incident & Deadlock Investigation Center, Blast Radius, Wait-For Graph, Strict 2PL & Forensic Replay

Q10. Performance & Benchmark Workload Lab: Live Execution, Step-by-Step Testing, Schema & Concepts

Q11. OS Simulation Lab: FCFS, SJF, SRTF, RR, Priority, Gantt Charts, Formulas, FIFO/LRU/Optimal & Banker's

Q12. DBMS Simulation Lab: ACID, Conflict Serializability & Precedence Graph, Strict 2PL & WAL Recovery

Q13. What-If Scenario Analysis Lab: Counterfactual Modeling, Formulas, Codebase & Schema

Q14. Overall Project Achievements & Exact Individual Contributions (Krishna, Aditya, Akanksha, Akshita)

Q15. Full System, Backend & Frontend Architecture, Complete OS/DBMS/CN Concept Matrix & API Pipeline

Appendix: Complete 32-Table Physical Schema vs. Phase-II 31-Table ER Diagram Reconciliation Map

SECTION 1 • PROJECT FOUNDATION, UNIQUENESS & CORE CONCEPTS

1. What is OS-DBX About, Why Is It Needed, How Are OS & DBMS Justified, What Is Its USP, and What Are the Correlation Engine & Identity Bridge?

30-SECOND VIVA PITCH (DIRECT ANSWER FOR EXAMINER)

OS-DBX ("Operating System – Database Cross-Layer Observability Platform") is a unified full-stack observability, forensic investigation, and simulation platform that bridges the **semantic gap** between the Operating System kernel (processes, threads, CPU/RAM, filesystem I/O, TCP sockets) and the Database Management System (MySQL queries, transactions, row/table locks, deadlocks, and WAL recovery). Using our custom **Identity Bridge** (which matches OS ephemeral TCP client sockets to MySQL server connections) and a deterministic **8-Factor Correlation Engine**, OS-DBX proves exactly *which OS process* caused *which database bottleneck or deadlock*, reconstructs the causal timeline, computes the **Blast Radius**, and allows safe **Forensic Replay** and **What-If Simulation**.

1.1 What Our Project Does & What Real-World Problem It Solves

- **The Real-World Problem ("The Observability Silo / Semantic Gap"):** In production systems, OS monitoring tools (like `top`, `htop`, Windows Task Manager, Prometheus Node Exporter, or `psutil`) only see OS Process IDs (`PID 14520` using 92% CPU or high disk I/O), but have **zero visibility** into what SQL transaction or lock that process is executing inside MySQL. Conversely, DBMS monitoring tools (like `SHOW PROCESSLIST`, MySQL Performance Schema, or MySQL Workbench) only see internal database identifiers (`THREAD_ID 89` or `CONNECTION_ID 44` waiting on an exclusive lock), without knowing which host application process, parent process tree, or OS user spawned it.
- **Why Naive Timestamp Matching Fails:** Most basic log aggregators correlate OS spikes and DB slow queries purely by checking if they happened at the same second. In our project, we prove why that causes **false positives** (e.g., a user watching YouTube in Chrome spiking CPU at the exact millisecond a background SQL query runs).
- **What OS-DBX Does:**
  1. **Collects Live Host OS Telemetry:** Process spawns/exits, parent-child trees, threads, CPU/RAM/Disk/Net usage, and real-time `watchdog` filesystem mutations.
  2. **Collects Live MySQL Telemetry:** Harvests statement execution history from `performance_schema.events_statements_history_long` (with automatic fallback to `events_statements_history` and `information_schema.processlist`).

3. **Bridges Identity Across Layers:** Deterministically links `OS PID ⇔ Local Ephemeral TCP Port ⇔ MySQL PROCESSLIST_ID ⇔ Transaction & Query ID`.
4. **Investigates & Simulates:** Detects live deadlocks via Wait-For Graph DFS cycle analysis, calculates cascading **Blast Radius** impact, performs **Forensic Incident Replay**, and provides interactive **OS & DBMS Simulation Labs** plus **What-If Scenario Modeling**.

1.2 What Is Meant by "OS-DBX: A Unified System for Tracking Computer & Database Problems"?

- **"Computer Problems" (OS Layer):** CPU starvation, memory thrashing/page faults, runaway child processes, unauthorized file modifications, disk seek saturation, and high process context switching.
- **"Database Problems" (DBMS Layer):** Unserializable concurrent schedules, lock contention convoys, circular wait deadlocks, long-running unindexed queries, and uncommitted state corruption during crashes.
- **"Unified System":** Instead of running separate OS and DB tools, OS-DBX normalizes both streams into a single **32-table 3NF MySQL relational schema (`os_dbx`)** exposed through a single **FastAPI REST backend**, a **10-module Rich CLI Console**, and a **Browser Frontend Playground**.

1.3 Software Classification & Real-World Applications

Dimension	Classification & Details
Type of Software	**Cross-Layer System Observability, APM/DBPM (Application & Database Performance Monitoring), Forensic Incident Response, and Algorithmic Simulation Platform.**
Real-World Applications	1. **Production DevOps & SRE Root-Cause Analysis:** Pinpointing which microservice worker PID triggered a MySQL row-lock convoy or CPU spike. 2. **Database Administration (DBA) Forensics:** Replaying historical deadlocks and evaluating blast radius across concurrent transactions. 3. **Security & Compliance Auditing:** Mapping Linux UIDs (`uid_linux`) and OS processes to specific SQL queries and file permission mutations. 4. **Academic & Engineering Laboratory:** Comparing live OS/DBMS behavior side-by-side with theoretical schedulers (FCFS/SJF/SRTF/RR), page replacement (FIFO/LRU/Optimal), Banker's algorithm, Strict 2PL, and ARIES WAL recovery.

1.4 How Both OS and DBMS Are Justified in Our Project

Operating System (OS) Justification:

- **Live Kernel/Host Inspection:** Process Control Block (PCB) tracking (PID, PPID, state, UID), kernel thread enumeration (`threads`), CPU/RAM/Disk/Network counters, and `watchdog` filesystem events.
- **Core OS Algorithms Implemented:** CPU Scheduling (FCFS, SJF, SRTF, Round Robin, Priority), Virtual Memory Page Replacement (FIFO, LRU, Optimal), Deadlock Avoidance (Banker's Algorithm & Resource Allocation Graph), and Disk Head Scheduling (SSTF).

Database Management System (DBMS) Justification:

- **3NF Relational Schema:** 32 normalized tables with 27+ foreign keys, composite unique watermarks, and B-Tree performance indexes.
- **Live DB Internals & Concurrency Engine:** Harvests MySQL `performance_schema` & `information_schema`, implements ACID transaction management, Strict Two-Phase Locking (Strict 2PL with Shared `S` / Exclusive `X` locks & lock upgrades), Wait-For Graph deadlock cycle detection, Conflict Serializability precedence graphs, and WAL UNDO/REDO crash recovery.

1.5 Uniqueness & USP (Unique Selling Proposition) of OS-DBX

1. **Zero-Instrumentation TCP Loopback Identity Bridge:** Maps OS PIDs to MySQL Connection IDs externally using OS socket tables (`netstat -ano -p tcp` + `psutil.net_connections`) joined with `information_schema.processlist.HOST` and `performance_schema.session_connect_attrs` — requiring zero modification to legacy application code.
2. **PID Recycling Guard & Null-PID Filesystem Honesty:** Verifies `process_start_time ≤ event.timestamp ≤ process_stop_time` so recycled OS PIDs never cause false correlations, and enforces `pid=None` (`TEMPORAL` classification) for user-space filesystem events rather than fabricating fake PIDs.
3. **Deterministic 8-Factor Evidence Scoring (No Black-Box Guessing):** Computes an auditable boolean checklist (`8/8` checks: PID, DB Connection, Transaction, Query, Timestamp Window ≤60s, Causal Ordering, Lock Relation, Host Match) instead of opaque probabilities.
4. **Dual "Live Reality + Controlled Simulation" Architecture:** Combines live host/MySQL telemetry with deterministic OS/DBMS simulation labs and counterfactual What-If modeling in one platform.

1.6 What Is Meant by "Identity Bridge" and "Correlation Engine"?

- **Identity Bridge (`backend/app/services/identity_bridge.py`):** The translation layer that answers "*Which OS Process ID owns which MySQL Connection ID right now?*" Every TCP connection between a local process and MySQL (`port 3306`) has a unique ephemeral client port (e.g., `127.0.0.1:54321 → 127.0.0.1:3306`). The OS knows that port `54321` belongs to `PID 18420`; MySQL's `information_schema.processlist` knows that client `localhost:54321` is `ID 42`. The Identity Bridge joins them on port `54321` to establish `PID 18420 ⇔ Connection 42`!
- **Correlation Engine (`backend/app/services/correlation_service.py`):** The rule-based causal verifier that takes normalized OS events and DBMS query/transaction events, applies 7 strict integrity guards (Host Guard, Duplicate ID Guard, Null-PID Guard, PID Recycling Guard, Transaction/Query Identity Guards), scores the relationship across 8 forensic factors via `EvidenceService`, and classifies the link as `DIRECT`, `TEMPORAL`, or `NONE (NOT CORRELATED)`, saving the causal chain into `cross_layer_traces` and `event_correlations`.

SECTION 2 • LANGUAGE CHOICE, FULL-STACK TECHNOLOGY STACK & NETWORK BINDING

2. Why Use Python, What Is the Full-Stack Tech Stack, and Why Does the Server Run Specifically on `127.0.0.1:8000`?

30-SECOND VIVA PITCH (DIRECT ANSWER FOR EXAMINER)

We selected **Python 3.12** because it is the only language that natively combines direct cross-platform OS kernel introspection (via `psutil` and `watchdog`), high-performance asynchronous REST APIs (via **FastAPI + Uvicorn + Pydantic**), pooled MySQL database drivers (`mysql-connector-python`), and rich terminal UI rendering (`Rich`) within a single unified codebase. Our FastAPI server runs on `127.0.0.1:8000` (IPv4 loopback) because our **Identity Bridge** relies on matching local TCP client sockets against MySQL (`127.0.0.1:3306`) on the same physical host without NAT translation, while also preventing unauthenticated external network access to privileged OS process and filesystem telemetry.

2.1 Why Python Was Chosen for OS-DBX

1. **First-Class OS Kernel & System Call Access:** Libraries like `psutil` (written in C/Cython under the hood for Windows Win32 API and Linux `/proc`) and `watchdog` (wrapping Windows `ReadDirectoryChangesW` and Linux `inotify`) allow microsecond-level inspection of PCBs, threads, sockets, and disk/network I/O counters.
2. **Modern Async Backend & Automatic OpenAPI Schema Validation:** `FastAPI` + `Pydantic v2` provide strict type validation, automatic Swagger UI documentation at `http://127.0.0.1:8000/docs`, and `asyncio` background lifespan tasks (`_consume_stream` and `_consume_dbms_poller`).
3. **Deterministic Algorithm Prototyping:** Clean dataclass and graph structures (`collections.deque`, DFS recursion stacks) make implementing CPU schedulers, page replacement, Banker's algorithm, Wait-For Graph cycle detection, and ARIES WAL recovery concise and verifiable across **100 automated pytest unit & integration tests**.

2.2 Complete Full-Stack Technology Stack (Backend, Frontend, CLI, Database & DevOps)

Architectural Layer	Technologies & Libraries Used	Exact Codebase Files	Role in OS-DBX
Backend API Gateway	**FastAPI 0.141.1**, **Starlette 1.6.0**, **Uvicorn 0.52.4** (ASGI), **Pydantic 2.13.5**, **pydantic-settings 2.15.0**	`backend/app/main.py`, `backend/app/api/routes_*.py`, `backend/app/config/settings.py`	Hosts 40+ REST endpoints, validates JSON payloads, runs background OS & DBMS ingestion daemons, serves OpenAPI docs.
Database & Storage Layer	**MySQL 8.0+ (InnoDB Engine)**, **mysql-connector-python 26.7.0**, `MySQLConnectionPool(pool_size=10)`, MySQL `performance_schema` & `information_schema`	`database/schema.sql`, `database/seed.sql`, `backend/app/config/database.py`, `backend/app/repositories/*.py`	Persists 32 normalized 3NF tables with ACID transactions, foreign keys, and row-level locking (`SELECT ... FOR UPDATE`).
Security & Authentication	**python-jose 3.5.0** (JWT `HS256`), **bcrypt 5.0.0** (salted hashing), FastAPI `OAuth2PasswordBearer`	`backend/app/core/auth.py`, `backend/app/api/routes_auth.py`, `backend/app/api/routes_audit.py`	Stateless Bearer JWT authentication (60-min expiry), bcrypt password verification, Linux UID-to-Admin mapping, and RBAC audit logging.
OS Collectors & Simulation Engine	**psutil 6.0.0**, **watchdog 6.0.0**, Python `threading`, `queue.Queue`, `subprocess` (`netstat`), `ctypes`	`os_monitor/adapters/windows_adapter.py`, `os_monitor/core/stream.py`, `os_monitor/simulations/**`	Captures live process/thread/resource/filesystem telemetry into a 10,000-event burst buffer and executes OS scheduling/memory/deadlock algorithms.
Interactive CLI Dashboard (Primary Demo UI)	**Rich 15.0.0** (Panels, Tables, Live Spinners, ANSI styling), **Requests 2.32.4** (`requests.Session()`)	`cli.py` (2,636 lines)	Full-featured 10-module interactive terminal observability console with a real-time 4-row Live Health & Device Monitor banner.
Web Frontend Playground	**HTML5, CSS3 (Custom Design System), Vanilla ES6 JavaScript** (Single-Page App architecture; planned React integration in Phase-III)	`index.html` (531 lines), `styles.css` (904 lines), `app.js` (766 lines) on branch `origin/front-end`	Browser-based interactive OS/DBMS learning playground featuring live backend health polling (`/health`, `/health/details`) and an interactive step-by-step CPU Scheduling Gantt visualizer.
Testing, CI & Diagnostics	**Pytest 9.0.3**, **FastAPI TestClient / HTTPX**, **GitHub Actions CI**	`backend/tests/*`, `os_monitor/tests/*`, `.github/workflows/ci.yml`, `tools/system_diagnostics.py`	100 automated unit & end-to-end integration tests verifying correlation guards, deadlocks, replay 409 conflicts, and simulations.

2.3 Which IP & Port Does Our Server Run On, and Why Only This Address?

- **FastAPI Backend Address:** `http://127.0.0.1:8000` (IPv4 Loopback IP `127.0.0.1`, TCP Port `8000`), configured in `cli.py:18` (`API_BASE_URL = "http://127.0.0.1:8000"`) and `app.js` on `origin/front-end`.

- **MySQL Server Address:** `localhost:3306` / `127.0.0.1:3306`, configured in `backend/app/config/settings.py:9-10`.
- **Three Critical Reasons Why `127.0.0.1:8000` Was Specifically Selected:**
  1. **Deterministic Loopback Socket Identity Matching (Core Technical Requirement):** In `IdentityBridgeService` (`backend/app/services/identity_bridge.py:40-115`), the backend correlates an OS process with a MySQL session by inspecting active TCP connections on the host machine (`127.0.0.1:<ephemeral_client_port> → 127.0.0.1:3306`) and matching that exact ephemeral port against `SELECT ID, HOST FROM information_schema.processlist`. Running on `127.0.0.1` guarantees that the client process, OS kernel socket table, FastAPI server, and MySQL server share the same loopback network namespace without NAT port translation or router rewriting.
  2. **Single-Host Causality Enforcement (Host Guard):** `CorrelationService.correlate()` (`correlation_service.py:93-98`) enforces a strict **Host Guard**: "*host_id values do not match; cross-host correlation is not permitted*". Keeping the server on `127.0.0.1` ensures that OS PIDs and timestamps originate from the same physical kernel clock.
  3. **Zero External Attack Surface & Port Non-Conflict:** Because OS-DBX exposes powerful system inspection routes (live process tables, filesystem paths, and benchmark workload triggers), binding to `127.0.0.1` instead of `0.0.0.0` prevents untrusted devices on public Wi-Fi/LAN from probing the machine. Port `8000` is the standard unprivileged ASGI development port (above the privileged `0–1023` range) that avoids collisions with MySQL (`3306`), React/Vite (`3000` / `5173`), or Windows IIS (`80` / `443`).

SECTION 3 • SECURITY, JWT AUTHENTICATION, LINUX UID MAPPING & USER REGISTRATION

3. What Are JWT Credentials, Why Did We Use Login, What Is Linux UID Mapping, and How Does "Register New User" Work in Code & Schema?

30-SECOND VIVA PITCH (DIRECT ANSWER FOR EXAMINER)

OS-DBX secures all observability, simulation, and benchmark APIs using **stateless JWT (JSON Web Token) Bearer authentication** signed with `HS256` and backed by **salted** `bcrypt` **password hashes** in the MySQL `users` table. Every user account in OS-DBX is mapped to a **Linux/POSIX User ID (** `uid_linux` **)** so that database entities (processes, files, permissions, simulations, and audit logs) are linked to an OS-level user identity in **3rd Normal Form (3NF)**. Furthermore, following POSIX kernel semantics where `UID 0` is `root`, our authentication engine automatically grants administrator privileges (`is_admin = True`) when `uid_linux == 0` .

3.1 What Are JWT Credentials & Why We Used the "Login" Feature

- **What JWT Is in OS-DBX:** A JSON Web Token is a compact, cryptographically signed Base64URL token (`header.payload.signature`) generated by `create_access_token()` in `backend/app/core/auth.py:37-46` using `ALGORITHM = "HS256"` and a 256-bit `SECRET_KEY` with an expiration time of **60 minutes** (`ACCESS_TOKEN_EXPIRE_MINUTES = 60`).
  - **Payload structure:** `{"sub": user.username, "exp": <utc_timestamp + 60m>}` .
  - **Transmission:** Stored in memory by `cli.py` and sent on every HTTP request via `get_headers()` (`cli.py:23-26`) as `Authorization: Bearer <access_token>` .
- **Why We Used the "Login" Feature & Its Significance:**
  1. **Protecting Privileged System Telemetry:** OS-DBX reads live host processes, file permissions, and MySQL query text, and can trigger stress benchmarks. Unauthenticated access would be a security vulnerability.
  2. **Role-Based Access Control (RBAC):** Distinguishes regular operators from Administrators (`is_admin = True`), restricting sensitive audit endpoints like `GET /api/audit/` (`backend/app/api/routes_audit.py:45-55`) exclusively to admins.
  3. **Non-Repudiation & Security Audit Trail:** Every CLI menu action calls `log_cli_action()` (`cli.py:31-38`) → `POST /api/audit/log` , recording the authenticated `user_id` , `username` , `action` , and `timestamp` in the `audit_logs` table.
  4. **Python 3.12 Native** `bcrypt` **Hardening (Git Commit** `eaca677` **):** Instead of the deprecated `passlib` wrapper (which breaks on Python 3.12), we implemented direct `bcrypt.gensalt()` , `bcrypt.hashpw()` , and `bcrypt.checkpw()` in `backend/app/core/auth.py:22-34` , and fixed a subtle FastAPI dependency lifecycle bug in `get_current_user()` (lines 98–105) by using an isolated DB connection so route handlers never inherit a closed cursor.

3.2 What Is Linux UID Mapping & Where It Connects Across the Schema

In Linux/UNIX operating systems, every user is identified in the kernel by a numeric **User Identifier (UID)** (e.g., `0` for `root` , `1000+` for standard users). Every Process Control Block (PCB) and every inode on disk stores this UID. In OS-DBX, the `users` table acts as the **3NF Identity Anchor** mapping `username ⇔ uid_linux ⇔ user_id (PK)` :

```
CREATE TABLE users (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(100) NOT NULL UNIQUE,
    uid_linux BIGINT NOT NULL UNIQUE,
    password_hash VARCHAR(255) NULL,
    is_admin BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

- **POSIX Root Equivalence Logic (** `backend/app/core/auth.py:67-69` **):** `user_dict["is_admin"] = bool(user_dict.get("is_admin") or uname in ("admin", "root") or uid == 0)` . Thus, registering or logging in with `uid_linux = 0` maps directly to POSIX superuser privilege!
- **Five Foreign-Key Connections in** `database/schema.sql` **:**
  1. `processes.user_id → users(user_id)` (`fk_process_user` , lines 29–31): Links every live OS process to its owning user.
  2. `files.owner_id → users(user_id)` (`fk_file_owner` , lines 105–107): Tracks file ownership in filesystem monitoring.
  3. `permissions.user_id → users(user_id)` (`fk_permission_user` , lines 135–137): Maps `READ` / `WRITE` ACLs to users.
  4. `simulations.created_by → users(user_id)` (`fk_simulations_created_by` , lines 148–150): Tracks which user ran an OS simulation.
  5. `audit_logs.user_id` (lines 455–462): Records every CLI/API action performed by that user.

3.3 How "Register New User" Works Step-by-Step (Codebase + Schema + Logic)

Step	Layer & File Location	Function / Endpoint & Exact Logic
Step 1: Startup Admin Seeding	`backend/app/main.py` (lines 44–69)	`seed_admin_account()` runs on FastAPI startup: ensures default admin `username='admin'` , `uid_linux=0` , `is_admin=1` , password `Admin@OSDBX2026!` (hashed with `bcrypt`) exists in `users` .
Step 2: CLI Input Validation	`cli.py` (lines 207–233 inside `login_menu()`)	When user selects `[2] Register New User` , CLI prompts via validated helpers: `ask_required_str("Enter new username:")` , `ask_required_int("Enter Linux user ID (UID):")` , and `ask_required_str("Enter new password:", is_password=True)` . Sends `POST /api/auth/register` .

Step 3: Pydantic Schema Validation	`backend/app/models/users.py` (lines 7–14)	`UserCreate(UserBase)` enforces `username: str = Field(..., min_length=3, max_length=100)`, `uid_linux: int`, and `password: str = Field(..., min_length=6)`.
Step 4: Route Handler & SQL Insert	`backend/app/api/routes_auth.py` (lines 30–64)	`register_user(user_in: UserCreate)` checks `get_user_by_username()` first (raising HTTP 400 if taken), hashes password via `get_password_hash(user_in.password)`, and runs: `INSERT INTO users (username, uid_linux, password_hash) VALUES (%s, %s, %s)`. Catches MySQL IntegrityError `errno == 1062` (duplicate `uid_linux` or `username`) and returns HTTP 400.

SECTION 4 • REAL-TIME TELEMETRY BANNER, CORE HEART OF OS-DBX & COMPETITIVE COMPARISON

4. What Is the "OS-DBX Live Health & Device Monitor", What Are Latency, Traces, and TXs, What Is the Main Heart of the Project, and Why Choose It Over Alternatives?

30-SECOND VIVA PITCH (DIRECT ANSWER FOR EXAMINER)

The **OS-DBX Live Health & Device Monitor** is a real-time 4-layer telemetry HUD rendered at the top of our CLI (`display_live_health_panel()` in `cli.py:40-71`) by polling `GET /api/performance/system-health` (`routes_performance.py:53-260`). In a single glance, it displays live OS hardware metrics, MySQL Performance Schema throughput/latency/transactions/locks, cross-layer correlation trace counts, and the observability agent's own CPU/RAM footprint. The **main heart of OS-DBX** is the triad of `IdentityBridgeService` + `CorrelationService` + the dual async ingestion daemons (`_consume_stream` & `_consume_dbms_poller` in `main.py`).

4.1 Anatomy of the OS-DBX Live Health & Device Monitor Banner

```
┌──────────────────── OS-DBX LIVE HEALTH & DEVICE MONITOR ────────────────────┐
│ SYSTEM:   CPU: 14.2% | MEMORY: 68.4% | DISK: 1.8 MB/s | OS EVENTS: 4.25/s    │
│ DBMS:     CONNECTED [LIVE] | QUERIES: 2.1/s | LATENCY: 1.84ms | TXS: 1 | LOCKS: 2 │
│ CORR:     AVG LATENCY: 1.2ms | TRACES: 18 (14 DIRECT, 4 TEMPORAL) | DROPPED: 0 │
│ OVERHEAD: 0.8% CPU | 48.2 MB RAM                                            │
└──────────────────────────────────────────────────────────────────────────────┘
```

Banner Row & Component	What It Means & How It Is Computed in `backend/app/api/routes_performance.py`
Row 1: `SYSTEM` (`CPU`, `MEMORY`, `DISK`, `OS EVENTS/s`)	• **CPU & MEMORY:** Live host CPU utilization via `psutil.cpu_percent(interval=None)` and RAM percentage via `psutil.virtual_memory().percent` (lines 64–68). • **DISK (MB/s):** Stateful differential throughput computed from `psutil.disk_io_counters()`: `Δ(read_bytes + write_bytes) / (1024² × Δt)` (lines 70–83). • **OS EVENTS/s:** Executes `SELECT COUNT(*) FROM os_events WHERE timestamp >= NOW() - INTERVAL 1 MINUTE` divided by `60.0` (lines 140–146).
Row 2: `DBMS` (`CONNECTED`, `QUERIES/s`, `LATENCY`, `TXS`, `LOCKS`)	• **CONNECTED [LIVE]:** Verifies `SELECT @@performance_schema` is enabled (lines 149–154). • **QUERIES/s:** Statements observed in the last 60s in `dbms_query_observations` divided by `60.0`. • **LATENCY (`avg_query_latency_ms`):** Average execution duration in milliseconds of SQL queries over the last 60s (`AVG(execution_time_ms)`), where each statement's latency is converted from MySQL Performance Schema picoseconds (`TIMER_WAIT / 1,000,000,000`). • **TXS (Active Transactions):** Count of currently uncommitted transactions: `SELECT COUNT(*) FROM transactions WHERE status = 'ACTIVE'` (lines 176–179). • **LOCKS:** Count of held/waiting locks in `locks WHERE status IN ('HELD', 'WAITING')` plus live InnoDB lock waits in `performance_schema.data_lock_waits` (lines 181–198).
Row 3: `CORR` (`AVG LATENCY`, `TRACES`, `DIRECT`, `TEMPORAL`, `DROPPED`)	• **AVG LATENCY (`avg_correlation_latency_ms`):** Mean time taken by the correlation engine to match and persist cross-layer events (`AVG(metric_value)` from `performance_records`). • **TRACES (`traces_count`):** Total end-to-end root-cause investigation envelopes stored in `cross_layer_traces` (`SELECT COUNT(*) FROM cross_layer_traces`), broken down into `DIRECT` (socket-verified matches where `correlation_method LIKE '%SOCKET%'`) vs. `TEMPORAL` (time-window correlations) in `event_correlations` (lines 201–215). • **DROPPED:** Number of OS events dropped due to buffer overflow on the 10,000-slot `LiveEventStream` queue (`os_monitor/core/stream.py`).
Row 4: `OVERHEAD` (`% CPU`, `MB RAM`)	• Self-instrumentation of the OS-DBX FastAPI backend process itself via `proc = psutil.Process(os.getpid())`, reporting `proc.cpu_percent()` and Resident Set Size `proc.memory_info().rss / 1024**2` (lines 235–237). Proves that our monitoring tool does not itself become a bottleneck!

4.2 What Is the "Main Heart" of the Project?

The **main heart of OS-DBX** consists of three tightly coupled backend components:

1. **`IdentityBridgeService` (`backend/app/services/identity_bridge.py`):** The live socket-to-session translator that maps OS Process IDs (`PID`) to MySQL `PROCESSLIST_ID`s via TCP port inspection and session handshake attributes.
2. **`CorrelationService` & `EvidenceService` (`backend/app/services/correlation_service.py` & `evidence_service.py`):** The deterministic causal reasoning engine that enforces the PID Recycling Guard, Host Guard, and 8-Factor Evidence Checklist.
3. **Dual Lifespan Background Ingestion Workers (`_consume_stream` & `_consume_dbms_poller` in `backend/app/main.py:82-366`):** The continuous async loops that drain live OS events every `0.05s` and poll MySQL Performance Schema every `1.0s`, automatically correlating and persisting traces in real time.

4.3 Why OS-DBX Over Existing Alternatives & What Special Insights We Found

Existing Tool / Alternative	Limitation of Alternative	How OS-DBX Solves It (What Special We Found)
OS Tools (`htop`, Task Manager, `psutil` scripts)	Show high CPU/RAM/Disk per `PID`, but cannot see inside MySQL to tell which SQL query or lock that PID is waiting on.	OS-DBX maps the `PID` across the loopback TCP socket directly to the active SQL query and lock inside MySQL.

DBMS Tools (MySQL Workbench, `SHOW ENGINE INNODB STATUS`)	Show internal DB thread IDs and lock waits, but lose the host OS process tree, thread count, and filesystem context.	OS-DBX links MySQL `PROCESSLIST_ID` back to the OS process tree, parent process, CPU/RAM spikes, and Linux UID.
Enterprise APM (Datadog, New Relic, Dynatrace)	Commercial, closed-source SaaS agents requiring heavy SDK instrumentation, kernel eBPF root agents, and cloud billing; lack educational algorithm sandboxes.	OS-DBX is 100% self-hosted, open-source, requires **zero application code changes**, works on unprivileged Windows/Linux via `netstat` / `psutil` fallbacks, and integrates interactive OS & DBMS simulation labs.

> **Special Technical Discovery During Development:** We discovered three real-world edge cases that break naive correlation systems: (1) **Short-lived connections** closing before the next OS poll (solved via `_recent_conn_to_pid` LRU caching and `performance_schema.session_connect_attrs` `_pid` fallback); (2) **Windows** `psutil.AccessDenied` on non-admin terminals when reading system sockets (solved via our unprivileged `netstat -ano -p tcp` regex parser in `identity_bridge.py:40-60`); and (3) **Null PIDs on user-space filesystem events** (solved by separating `DIRECT` socket correlation from `TEMPORAL` file-path correlation).

SECTION 5 • CLI MODULE [1]: OS-DBX UNIFIED CROSS-LAYER OBSERVABILITY CONSOLE

5. What Is the "OS-DBX Unified Cross-Layer Observability Console", How Does It Work, and How Do "Inspect Active Cross-Layer Sockets" & "Execute Ad-Hoc SQL Query & Trace Causality" Work?

30-SECOND VIVA PITCH (DIRECT ANSWER FOR EXAMINER)

The **Unified Cross-Layer Observability Console** (Main Menu Option `[1]`, `unified_live_cross_layer_console()` in `cli.py:842-890`) is the flagship interactive command center of OS-DBX. Unlike separate OS or DB monitors, it renders a live side-by-side feed joining host OS events with MySQL Performance Schema queries in real time, provides a negative-control evaluator (proving why a browser playing YouTube is *not* falsely correlated with DB queries), inspects live TCP loopback sockets (`PID ↔ Port ↔ MySQL Connection ID`), and lets the examiner execute any ad-hoc SQL query while generating a **5-Panel Forensic Causality Proof** on the spot.

5.1 Connected Codebase Files & Database Schema Tables

- **CLI Functions (`cli.py`):** `unified_live_cross_layer_console()` (lines 842–890), `run_live_streaming_observatory()` (lines 542–626), `run_real_world_youtube_evaluator()` (lines 628–698), `run_identity_bridge_live_map()` (lines 1216–1274), `run_live_adhoc_sql_causality()` (lines 700–840).
- **Backend Routes & Services:** `backend/app/main.py` (background workers `_consume_stream` & `_consume_dbms_poller`), `backend/app/api/routes_bridge.py` (`GET /api/bridge/live-map`), `backend/app/api/routes_dbms_events.py` (`POST /api/dbms-events/collect`, `GET /api/dbms-events/observations`), `backend/app/api/routes_events.py` (`GET /api/events`), `backend/app/api/routes_correlation.py` (`POST /api/correlations/auto`), `backend/app/services/identity_bridge.py`, `backend/app/services/correlation_service.py`.
- **Schema Tables Involved:** `processes`, `os_events`, `dbms_query_observations`, `transactions`, `query_executions`, `cross_layer_traces`, `event_correlations`, and `performance_records`.

5.2 Detailed Breakdown of All 4 Sub-Features in Module [1]

Sub-Feature [1]: Live Streaming Cross-Layer Observatory (`run_live_streaming_observatory()`, `cli.py:542-626`)

- Runs a continuous `1.5s` refresh loop until `Ctrl+C`. Each cycle triggers `POST /api/dbms-events/collect`, fetches the latest 8 DBMS observations, latest 8 OS events, and the live socket map from `GET /api/bridge/live-map`.
- **Three-Tier Pairing Logic:**
  1. `DIRECT [SOCKET]`: If the DBMS query's `connection_id` is present in `bridge_map`, it resolves the exact `bridged_pid`, pairs it with that PID's OS event (or active PCB status), and marks it in bold green.
  2. `TEMPORAL`: If no socket match exists, it checks if an unmatched OS event occurred within `|t_db - t_os| ≤ 5.0 seconds` and pairs them by timestamp proximity (never by arbitrary list index).
  3. `NOT CORRELATED`: Unmatched OS or DBMS events outside the 5s window or lacking socket identity are displayed on separate rows as independent activity.

Sub-Feature [2]: Real-World Activity Evaluation — Browser/YouTube vs. DBMS Workload (`cli.py:628-698`)

- **Why It Exists:** Examiners often ask: *"If I am watching YouTube on Chrome while MySQL runs a query at the exact same millisecond, will your system falsely blame Chrome for the database query?"*
- **How It Works:** Scans live host processes via `psutil.process_iter(['pid', 'name'])` for browsers (`chrome.exe`, `msedge.exe`, `firefox.exe`, `brave.exe`). Queries `GET /api/bridge/live-map` and checks `any(pid in bridge_map.values() for pid in browser_pids)`. Because the browser has **zero TCP sockets connected to port 3306**, OS-DBX proves in a 4-stage forensic output that the co-occurring activity is strictly `TEMPORALLY_RELATED` / `NONE`, eliminating false positives!

Sub-Feature [3]: "Inspect Active Cross-Layer Sockets & Process Identity Map" (`cli.py:1216-1274`)

- **Codebase Connection:** Calls `GET /api/bridge/live-map` in `backend/app/api/routes_bridge.py:9-39`, which invokes `IdentityBridgeService.build_pid_to_connection_map()` (`identity_bridge.py:90-146`) and enriches each mapped OS PID with `psutil.Process(pid).name()`, `status()`, and `username()`.
- **Two Interactive Modes:**
  1. **Mode `[1]` Instant Passive Scan:** Captures all currently open TCP connections between host OS processes and MySQL port 3306.
  2. **Mode `[2]` Dynamic Live Mapping Test (Recommended for Viva Demo!):** Because MySQL connections from pooled REST calls finish in milliseconds, Mode `[2]` opens a real `mysql.connector.connect()` connection directly from the CLI process, keeps the TCP socket open across a `SELECT SLEEP(0.05)` query while simultaneously calling `GET /api/bridge/live-map`, and prints the live table showing `OS PID ⇔ Process Name (python.exe) ⇔ OS User ⇔ MySQL Connection ID ⇔ Status (ESTABLISHED [LIVE])`.

Sub-Feature [4]: "Execute Ad-Hoc SQL Query & Trace Causality (Interactive Proof)" (`cli.py:700-840`)

- **Why It Is Needed:** Provides an indisputable, live end-to-end proof during the Viva that any SQL query typed by the examiner can be traced from the OS process down to the TCP socket, into MySQL Performance Schema, and back into a persisted cross-layer correlation trace.
- **Exact Step-by-Step Code Logic (`run_live_adhoc_sql_causality()`):**
  1. **Input & Safety Guard (lines 708–718):** Prompts for any SQL query (default `SELECT * FROM users LIMIT 3`) and blocks destructive DDL (`drop database`, `drop table`, `truncate table`, `shutdown`, `grant all`).
  2. **OS Process Snapshot (lines 720–730):** Captures the caller's own OS PID via `caller_pid = os.getpid()` and queries `psutil.Process(caller_pid)` for `name()`, `username()`, `cpu_percent()`, and `memory_percent()`.

3. **Live Socket & Ephemeral Port Extraction (lines 736–753):** Opens a real MySQL connection, reads `conn_id = conn.connection_id`, and scans `psutil.net_connections(kind='tcp')` for `c.pid == caller_pid and c.raddr.port == 3306` to capture the exact ephemeral TCP client port (e.g., `127.0.0.1:58412 → 127.0.0.1:3306`).
4. **Query Execution & Performance Schema Harvest (lines 755–789):** Executes the SQL statement, times it in milliseconds, triggers `POST /api/dbms-events/collect`, locates the newly harvested `observation_id` in `dbms_query_observations`, and triggers `POST /api/correlations/auto`.
5. **Renders 5 Pedagogical Causality Panels (lines 791–839):**
   - **Panel 1 (`1. WHAT HAPPENED IN OS?`):** Process Name, OS PID, User Context, CPU%, Memory%, Action.
   - **Panel 2 (`2. WHAT HAPPENED IN DBMS?`):** MySQL Thread/Conn ID, SQL Executed, Execution Time (ms), Rows Returned, Observation ID.
   - **Panel 3 (`3. HOW WERE THEY CONNECTED?`):** Visualizes the bridge: `OS Process (PID) → Local TCP Socket (127.0.0.1:<port>) → MySQL Server Port (3306) → DBMS Connection ID`.
   - **Panel 4 (`4. WHAT EVIDENCE PROVES THE CONNECTION?`):** Checklist of 5 verified proofs (OS Process Exists, Active TCP Socket Bound, DBMS Session Handshake, Temporal Alignment, Query Text Fingerprint) + classification `DIRECTLY_CORRELATED`.
   - **Panel 5 (`5. WHAT HAPPENED NEXT?`):** Confirms persistence across `dbms_query_observations`, `os_events`, and `cross_layer_traces` / `event_correlations`.

5.3 OS, DBMS & CN Concepts Used in This Console

- `OS CONCEPTS` Process Control Block (PCB) attributes, system call execution, user-space vs. kernel-space context, process resource utilization sampling.
- `CN CONCEPTS` TCP/IP 4-tuple socket addressing (`Source IP, Ephemeral Source Port, Destination IP, Destination Port 3306`), `ESTABLISHED` TCP state machine, loopback interface (`127.0.0.1`).
- `DBMS CONCEPTS` Client connection lifecycle (`PROCESSLIST_ID`), MySQL Performance Schema statement instrumentation, picosecond timer conversion, deterministic trace persistence.

SECTION 6 • CLI MODULE [2]: OS MONITORING & LIVE OBSERVATORY

6. Is "OS Monitoring & Live Observatory" Really Live on the Device? What Are Its Pros/Cons, Codebase, Schema Tables, "View Real-Time Process Table" Logic, and Step-by-Step Testing Procedure?

30-SECOND VIVA PITCH (DIRECT ANSWER FOR EXAMINER)

Yes — it is 100% live on the host machine. It inspects the real Windows/Linux kernel in real time using `psutil` and `watchdog` via two parallel pathways: (1) a continuous background producer-consumer pipeline (`WindowsAdapter` pushing into the 10,000-slot `LiveEventStream` queue, drained every `0.05s` by `_consume_stream()` in `main.py` into 7 normalized MySQL tables), and (2) on-demand live snapshot collectors triggered by REST API calls.

6.1 Advantages & Disadvantages (Architectural Trade-Offs)

Advantages (Pros):

- **Zero Kernel Driver / Root Requirement:** Runs safely in standard user space on Windows and Linux via `psutil` and `watchdog` without requiring custom C kernel modules or eBPF root privileges.
- **Burst-Safe Non-Blocking Queue:** `LiveEventStream` (`maxsize=10000` in `os_monitor/core/stream.py`) buffers sudden event spikes so OS monitoring never blocks on MySQL disk writes.
- **Self-Healing Lifecycle:** `routes_os_events.py` includes `ensure_adapter_started()` to automatically restart collector threads if stopped.
- **Rich Metadata Capture:** Captures threads (`proc.threads()`), parent-child process trees (up to 10 levels), I/O counters, and POSIX octal file permissions (`oct(st_mode)[-3:]`).

Disadvantages / Limitations (Cons):

- **Polling Resolution Window (1.0s – 2.0s):** Because process creation is polled via `psutil.process_iter()` every 1–2 seconds rather than hooking kernel `fork()` / `execve()` syscalls, ultra-short-lived processes (<1s lifetime) can finish between polls.
- **User-Space Filesystem Attribution (`pid=None`):** Standard user-space filesystem watchers (`watchdog` / `ReadDirectoryChangesW`) report *which file path* changed, not *which PID* wrote to it. OS-DBX honestly records `pid=None` and uses `TEMPORAL` correlation.
- **OS Privilege Boundaries:** Inspecting thread lists or I/O counters of elevated `SYSTEM` daemons raises `psutil.AccessDenied` (gracefully caught and skipped).

6.2 Codebase Architecture & Schema Tables for OS Monitoring

- **Collector Files:**
  - `os_monitor/adapters/windows_adapter.py` (lines 9–201): Continuous daemon running `_monitor_processes()` , `_monitor_resources()` , and `_start_filesystem_monitor()` .
  - `os_monitor/collector.py` (lines 1–95): `snapshot_processes()` , `diff_snapshots()` , and `build_event()` .
  - `os_monitor/resource_monitor.py` (lines 1–105): `collect_metrics()` with noise filters (ignores Windows `PID 0` System Idle Process and idle processes below `CPU_THRESHOLD = 1.0%` and `MEMORY_THRESHOLD = 0.5%`).
  - `os_monitor/filesystem_monitor.py` (lines 1–90): Recursive `watchdog.observers.Observer` handler for `file_created` , `file_modified` , `file_deleted` , and `file_moved` .
  - `backend/app/api/routes_os_events.py` (lines 1–463) & `backend/app/api/routes_events.py` (lines 1–64).
- **Physical SQL Tables in `database/schema.sql` (9 OS Tables):**
  1. `users` (`user_id` , `username` , `uid_linux`)
  2. `processes` (`process_id` PK, `pid` UNIQUE, `ppid` , `user_id` FK, `name` , `state` , `cpu_percent` , `memory_percent` , `start_time` , `end_time`)
  3. `threads` (`thread_id` PK, `process_id` FK, `tid` , `state` , `created_at`)
  4. `system_calls` (`syscall_id` PK, `process_id` FK, `syscall_name` , `arguments` , `return_value` , `timestamp`)
  5. `os_events` (`event_id` PK, `pid` NULL, `event_type` , `file_path` , `timestamp` , `details` JSON)
  6. `resource_metrics` (`metric_id` PK, `process_id` , `cpu_usage` , `memory_usage` , `disk_io` , `network_io` , `recorded_at`)
  7. `files` (`file_id` PK, `file_path` UNIQUE, `owner_id` FK, `permissions` , `size_bytes`)
  8. `directories` (`directory_id` PK, `path` UNIQUE, `parent_directory_id` Self-FK)
  9. `permissions` (`permission_id` PK, `file_id` FK, `user_id` FK, `access_type` , `granted_at`)

  (Note on Phase-II Report ER Diagram: In the conceptual ER diagram, `resource_usage` maps to physical table `resource_metrics` , and `filesystem_events` / `network_events` are unified inside `os_events` via `event_type` and `resource_metrics.network_io` .)

6.3 Deep Dive: "View Real-Time Process Table & Lifecycle Status" (Sub-Option [2])

- **Where Written:** CLI function `view_live_process_table()` (`cli.py:273-305`) → calls `GET /api/os-events/processes?limit=30` in `backend/app/api/routes_os_events.py:366-417` → invokes `snapshot_processes()` in `os_monitor/collector.py:42-58` .
- **Exact Logic Behind It:**

1. `snapshot_processes()` iterates over all live kernel processes via `psutil.process_iter(["pid", "ppid", "name", "username", "exe", "status"])`.
2. For each active process, `routes_os_events.py` resolves or inserts the OS `username` into the `users` table and upserts the process into MySQL:

```
INSERT INTO processes (pid, ppid, user_id, name, state, cpu_percent, memory_percent, start_time)
VALUES (%s, %s, %s, %s, %s, %s, %s, %s)
ON DUPLICATE KEY UPDATE
    ppid = VALUES(ppid), user_id = VALUES(user_id),
    name = VALUES(name), state = VALUES(state),
    cpu_percent = VALUES(cpu_percent), memory_percent = VALUES(memory_percent);
```

3. It then queries the top 30 active processes joined with `users` and renders a Rich table showing `PID`, `PPID`, `Process Name`, `User`, `State` (`running`, `sleeping`, `stopped`, `zombie`), and `Start Time`.

- **OS Concepts Used: Process Control Block (PCB)** attributes, **5-State Process Lifecycle Model** (New/Spawned → Ready/Running → Waiting/Sleeping → Terminated), **Parent-Child Process Hierarchy** (`PID` and `PPID` via `fork()` / `CreateProcess()`), and **Kernel Thread Enumeration**.

6.4 Step-by-Step Live Testing Instructions for All 6 Sub-Features of Module [2]

1. From Main Menu, press `2` to open **OS Monitoring & Live Observatory**.
2. **Test `[1]` (View Live OS Event Stream):** Press `1`. Calls `GET /api/events?limit=25` and displays live `process_created`, `process_terminated`, and resource spike events from `os_events`.
3. **Test `[2]` (View Real-Time Process Table & Lifecycle Status):** Press `2`. Calls `GET /api/os-events/processes?limit=30`, snapshots live host processes via `psutil`, persists them to `processes`, and displays PID, PPID, User, Name, and Lifecycle State.
4. **Test `[3]` (View Process Resource Usage Metrics):** Press `3`. Calls `GET /api/os-events/resources?limit=25`, runs `collect_metrics()` (filtering out idle noise), writes to `resource_metrics`, and displays `CPU %`, `Memory %`, and `Disk I/O (Bytes)`.
5. **Test `[4]` (View Filesystem Monitoring Events):** Press `4`. Automatically verifies collector status via `GET /api/os-events/status` (auto-starting via `POST /api/os-events/start` if needed) and lists real-time Watchdog file events from `GET /api/os-events/filesystem`.
6. **Test `[5]` (Trigger Live Filesystem Change Test & Observe Detection):** Press `5`. The CLI physically creates `os_dbx_live_watch_test.tmp` on disk, writes verification text, dispatches the event, waits `0.5s`, queries `GET /api/os-events/filesystem?limit=5` to prove the exact `file_created` event was captured in MySQL, and cleans up the file!
7. **Test `[6]` (View OS Live Stream Collector Metrics & Buffer Health):** Press `6`. Calls `GET /api/os-events/metrics` and displays `events_received`, `events_processed`, `events_dropped`, current queue depth out of `10000`, and adapter health (`RUNNING`).

SECTION 7 • CLI MODULE [3]: DBMS MONITORING & LIVE QUERY OBSERVATORY

7. What Is the "DBMS Monitoring & Live Query Observatory", Why Is It Needed, How Does It Work in Code & Schema, and What DBMS Concepts Are Used in Depth?

30-SECOND VIVA PITCH (DIRECT ANSWER FOR EXAMINER)

The **DBMS Monitoring & Live Query Observatory** (Main Menu Option `[3]`, `dbms_monitoring_menu()` in `cli.py:1024-1062`) non-invasively harvests executed SQL statements, execution latencies, row counts, and connection IDs directly from MySQL's internal `performance_schema` engine via `MySQLQueryCollector` (`backend/app/collectors/mysql_query_collector.py`). It features a **3-tier self-healing fallback**, converts hardware timer waits from **picoseconds to milliseconds**, deduplicates observations via composite watermarks, and feeds live query telemetry into both `dbms_query_observations` and `performance_records`.

7.1 Why This Feature Is Needed in OS-DBX

- External applications, ORMs, and database clients execute SQL queries directly against MySQL without routing them through our REST transaction endpoints.
- To achieve true observability, OS-DBX must capture **every SQL query executed on the MySQL server** along with its exact `THREAD_ID`, `PROCESSLIST_ID (connection_id)`, execution duration in milliseconds, rows affected, and completion status — without requiring slow general query logs on disk or proxy middleware.

7.2 Step-by-Step Internal Working of `MySQLQueryCollector` (`mysql_query_collector.py`)

1. **3-Tier Self-Healing Collection Strategy (`poll_once()`, lines 148–206):**
   - **Tier 1 (Primary):** Queries `performance_schema.events_statements_history_long AS statements JOIN performance_schema.threads AS threads ON threads.THREAD_ID = statements.THREAD_ID` (`STATEMENT_QUERY_TEMPLATE`, lines 12–43).
   - **Tier 2 (Auto-Consumer Activation & Thread History Fallback):** If `events_statements_history_long` is disabled, `_ensure_consumers_enabled()` (lines 84–94) automatically executes: `UPDATE performance_schema.setup_consumers SET ENABLED = 'YES' WHERE NAME IN ('events_statements_history_long', 'events_statements_history')` and falls back to `performance_schema.events_statements_history`.
   - **Tier 3 (Processlist Fallback):** If `performance_schema` is disabled at the server level, `poll_from_processlist()` (lines 96–146) queries `information_schema.processlist WHERE COMMAND <> 'Sleep' AND INFO IS NOT NULL`.
2. **Self-Observation Noise Filtering (lines 29–40):** Filters out OS-DBX's own monitoring connection (`threads.PROCESSLIST_ID <> CONNECTION_ID()`) and internal polling queries (`NOT LIKE '%dbms_query_observations%'`, `'%events_statements_history%'`, etc.) so the collector never creates an infinite feedback loop.
3. **Picosecond-to-Millisecond Latency Conversion (line 172):** MySQL Performance Schema measures `TIMER_WAIT` in **picoseconds (10^{-12} s)**. Since 1 ms = 10^9 ps, the collector computes:

   $$\text{execution_time_ms} = \text{round}(\text{float}(\texttt{TIMER_WAIT}) / 1{,}000{,}000{,}000, 4)$$
4. **Two-Level Idempotent Deduplication:**
   - **In-Memory Watermark:** `self._seen_event_ids[source_thread_id] = max_event_id` skips statements already processed in the current session.
   - **Database Unique Constraint:** `DBMSObservationRepository.insert_many()` (`dbms_observation_repository.py:10-71`) uses `ON DUPLICATE KEY UPDATE` on `uq_dbms_query_source (source_thread_id, source_event_id)`. For each newly inserted row, it also writes `dbms.query.execution_time_ms` and `dbms.query.rows_affected` (with `data_source='LIVE_MYSQL'`) into `performance_records`!

7.3 All 3 Sub-Features in CLI Module [3] & How to Test Them Live

Sub-Feature in `cli.py`	Function & API Route	What It Does & How to Test Live
`[1]` **View Recent Live DBMS Observations**	`fetch_dbms_observations()` (`cli.py:895-941`) → `GET /api/dbms-events/status` & `GET /api/dbms-events/observations?limit=15`	Checks if MySQL `performance_schema` is online, then displays a Rich table with `Obs ID`, `Conn ID`, `Type` (`SELECT` / `INSERT` / `UPDATE`), `Time (ms)`, `Rows`, `Status`, and `SQL Text`.
`[2]` **Poll MySQL Performance Schema (Collect New Events)**	`poll_performance_schema()` (`cli.py:943-956`) → `POST /api/dbms-events/collect` (`{"limit": 25, "persist": True}`)	Triggers an immediate harvest from MySQL `performance_schema`, persists new observations into `dbms_query_observations` and `performance_records`, and prints the count collected & persisted.
`[3]` **Execute Live User SQL Query & Observe Capture**	`execute_live_user_query()` (`cli.py:958-1022`) → Direct `mysql.connector` query + `POST /api/dbms-events/collect`	**Live Test:** Press `3`, type `SELECT username, uid_linux FROM users;`, and press Enter. The CLI executes the query against MySQL, displays the returned rows, immediately polls Performance Schema, and shows your exact query captured as a new `observation_id`!

7.4 Schema Tables & Deep DBMS Concepts Used

- **Schema Tables:** `dbms_query_observations` (lines 412–428), `query_executions` (lines 264–282), `transactions` (lines 236–247), and `performance_records` (lines 397–408). *(In the Phase-II ER Diagram, conceptual tables `databases`, `queries`, and `query_statistics` map directly to `dbms_query_observations`, `query_executions`, and `performance_records`.)*

- **DBMS Concepts Used in Depth:**
  1. **Internal Telemetry Ring Buffers (MySQL Performance Schema):** Non-blocking in-memory instrumentation tables (`events_statements_history_long`) that record statement lifecycle events without disk I/O lock contention.
  2. **Internal Thread ID vs. Client Connection ID Mapping:** MySQL assigns an internal engine `THREAD_ID` to background and foreground threads, while client sessions use `PROCESSLIST_ID` (`CONNECTION_ID()`). Joining `performance_schema.events_statements_history_long` with `performance_schema.threads` bridges internal instrumentation to external TCP connections.
  3. **Idempotent Event Ingestion & Composite Natural Keys:** Using `UNIQUE (source_thread_id, source_event_id)` guarantees that repeated polling never duplicates telemetry records.

SECTION 8 • CLI MODULE [4]: REAL-TIME CROSS-LAYER OS-DBMS CORRELATION ENGINE & IDENTITY BRIDGE

8. What Is the Real-Time Cross-Layer OS-DBMS Correlation Engine, What Is Its Motivation, Working, Codebase, Schema, Identity Bridge Map, and What OS, DBMS & CN Concepts Are Used?

30-SECOND VIVA PITCH (DIRECT ANSWER FOR EXAMINER)

The **Real-Time Cross-Layer OS-DBMS Correlation Engine** (Main Menu Option `[4]`, `cross_layer_correlation_menu()` in `cli.py:1303-1350`) is the analytical brain of OS-DBX. Its motivation is to replace unreliable "same-timestamp" guessing with **deterministic, socket-verified causality**. It combines `IdentityBridgeService` (which maps OS PIDs to MySQL Connection IDs via TCP loopback sockets and session handshake attributes) with `CorrelationService` and `EvidenceService` (which enforce 7 strict validation guards and compute an 8-factor forensic score to classify relationships as `DIRECT`, `TEMPORAL`, or `NONE`).

8.1 How the Identity Bridge Map Works in Code (`backend/app/services/identity_bridge.py`)

`IdentityBridgeService.build_pid_to_connection_map()` (lines 90–146) resolves `OS_PID ⇔ MySQL PROCESSLIST_ID` using a **3-Pronged Resolution Architecture**:

```
[OS Process: python.exe (PID 18420)]
       │  Opens TCP Client Socket: 127.0.0.1:54321 ──► 127.0.0.1:3306
       ▼
┌──────────────────────────────────────────────────────────────────────┐
│ PRONG 1: OS Kernel TCP Socket Table (get_os_client_ports, lines 62-88)
│  • Windows unprivileged fallback: `netstat -ano -p tcp` regex parser
│  • Cross-platform API: `psutil.net_connections(kind='tcp')`
│  • Extracts dictionary: client_ports[54321] = 18420 (Port ──► OS PID)
└──────────────────────────────────┬───────────────────────────────────┘
                                   │ Joins on Ephemeral Port (54321)
┌──────────────────────────────────▼───────────────────────────────────┐
│ PRONG 2: MySQL Processlist Inspection (lines 99-115)
│  • Executes: `SELECT ID, HOST FROM information_schema.processlist`
│  • Parses `HOST = "localhost:54321"` ──► port = 54321, ID = 42
│  • Links: pid_map[18420] = 42  AND caches _recent_conn_to_pid[42] = 18420
└──────────────────────────────────┬───────────────────────────────────┘
                                   │ Supplemented by
┌──────────────────────────────────▼───────────────────────────────────┐
│ PRONG 3: MySQL Session Connect Attributes Handshake (lines 116-136)
│  • Executes: `SELECT PROCESSLIST_ID, ATTR_VALUE FROM
│    performance_schema.session_connect_attrs WHERE ATTR_NAME = '_pid'`
│  • Verifies `psutil.pid_exists(c_pid)` and populates pid_map & LRU cache
└──────────────────────────────────────────────────────────────────────┘
```

8.2 How `CorrelationService` & `EvidenceService` Work (`correlation_service.py` & `evidence_service.py`)

1. **Seven Strict Integrity Guards in `CorrelationService.correlate()` (lines 64–201):**
   - **Guard 1 (Empty Input Guard):** Rejects empty event lists.
   - **Guard 2 (Duplicate Event ID Guard, line 88):** Ensures all `os_event_id`s in a correlation batch are unique.
   - **Guard 3 (Host Guard, lines 93–98):** Rejects correlation if `len(host_ids) > 1` (events from different machines can never be correlated).
   - **Guard 4 (Null-PID Filesystem Path, lines 101–103 & 203–268):** If all events have `pid is None` (Watchdog filesystem events), verifies they share a single non-empty `file_path` and span `< 60.0s` → classifies as `TEMPORAL` (`FILE_PATH_HOST_TIMESTAMP_WINDOW`).
   - **Guard 5 (Mixed Null/Non-Null PID Guard, lines 106–111):** Rejects mixing null-PID and non-null-PID events in the same call.
   - **Guard 6 (PID Recycling Guard, lines 114–146):** Verifies all events share the same positive `pid` AND that every `event.timestamp` falls strictly within `[process_start_time, process_stop_time]`. If an event occurred before `process_start_time` or after `process_stop_time`, it rejects the match as *"possible PID recycling"*!
   - **Guard 7 (DBMS Transaction & Query Identity Guard, lines 149–159):** Verifies matching non-null `transaction_id` and `query_id`. When all guards pass, classifies as `DIRECT` (`PID_TRANSACTION_QUERY_TIMESTAMP_ORDER`).
2. **Deterministic 8-Factor Evidence Score (`EvidenceService.evaluate()` in `evidence_service.py`):** Evaluates 8 boolean forensic checks: (1) `pid_match`, (2) `db_connection_match`, (3) `transaction_match`, (4) `query_match`, (5) `timestamp_relation` (≤60s), (6) `event_ordering` ($t_{OS} \leq t_{DBMS}$), (7) `lock_relation`, and (8) `host_match`. Persists `confidence_score = round(factors_met / 8, 2)` into `event_correlations`.

8.3 All 4 Sub-Features in CLI Module [4]

- `[1]` **Execute Real-Time Event Correlation (`run_cross_layer_correlation()`, `cli.py:1067-1144`):** Supports both **Mode 1 (Live System Telemetry)** (selecting real rows from `os_events` and `dbms_query_observations`) and **Mode 2 (Controlled Verification Input)** via `POST /api/correlations`.
- `[2]` **Run DBMS-OS Auto-Correlation Engine (`run_auto_correlation()`, `cli.py:1146-1214`):** Given any `observation_id` from `dbms_query_observations` and a time window (e.g. `5000ms`), calls `POST /api/correlations/auto` (`routes_correlation.py:181-267`), which uses `IdentityBridgeService.get_pid_for_connection()` + `AutoCorrelationRepository.find_os_events_in_window()` to automatically find and persist the matching OS events and trace!
- `[3]` **View Live Identity Bridge Map (`run_identity_bridge_live_map()`, `cli.py:1216-1274`):** Passive scan or dynamic live socket test via `GET /api/bridge/live-map`.

- `[4]` **Run Engine Verification Diagnostic (** `run_correlation_demo_cli()` **,** `cli.py:1275-1301` **):** Calls `GET /api/demo/correlation` (`routes_demo.py:11-93`) to run a live proof comparing a valid `DIRECT` case against a rejected **PID Recycling** case (`NONE`).

8.4 Schema Tables & OS / DBMS / CN Concepts Used in This Block

- **Physical Tables in** `database/schema.sql` **:** `cross_layer_traces` (`trace_id` PK, `pid` FK, `transaction_id` FK, `status` , `started_at` , `ended_at` , `summary`) and `event_correlations` (`correlation_id` PK, `trace_id` FK, `os_event_id` FK, `db_event_id` , `query_id` FK, `correlation_method` , `confidence_score` , `sequence_order`). *(In the Phase-II ER Diagram, conceptual tables* `process_connections` *and* `identity_bridge` *represent the live* `IdentityBridgeService` *mapping persisted into* `cross_layer_traces` *, while* `trace_steps` *maps to* `event_correlations` *.)*
- OS CONCEPTS **PID Allocation & PID Recycling (Wrap-around)** by the OS kernel; **Process Lifespan Windowing** (`create_time` to termination); **User-Space vs. Kernel VFS Filesystem Event Attribution**.
- DBMS CONCEPTS **Connection Session Attributes** (`performance_schema.session_connect_attrs`), **Processlist State Inspection**, and **Referential Integrity across Cross-Layer Entities**.
- CN CONCEPTS **Transport Layer (TCP) Socket Multiplexing**, **Ephemeral Client Port Allocation (IANA range 49152–65535)**, **Loopback Routing (** `127.0.0.1` **)**, and **TCP Connection State Inspection (** `ESTABLISHED` **)**.

SECTION 9 • CLI MODULE [5]: INCIDENT & DEADLOCK INVESTIGATION CENTER

9. What Is the "Incident & Deadlock Investigation Center", What Are Locks, Deadlocks, Wait-For Graphs, Blast Radius, and Forensic Execution Replay, and How Do They Work in Code & Schema?

30-SECOND VIVA PITCH (DIRECT ANSWER FOR EXAMINER)

The **Incident & Deadlock Investigation Center** (Main Menu Option `[5]`, `incident_investigation_menu()` in `cli.py:1534-1590`) is OS-DBX's post-mortem and live concurrency forensics suite. It detects live database deadlocks by constructing a directed **Wait-For Graph (WFG)** over active `HELD` and `WAITING` locks using Depth-First Search (DFS), maps the deadlocked transactions back to their owning **OS PIDs**, computes the cascading **Blast Radius** (`DIRECT`, `INDIRECT`, and `POTENTIAL` impact), reconstructs a chronological cross-layer timeline, and saves immutable **Forensic Execution Replays** (while enforcing an **HTTP 409 Conflict guard** if an incident lacks a correlated trace).

9.1 Theoretical Depth: Transactions, Locks, Deadlocks & Wait-For Graph (WFG)

- **What Is a Transaction?** A logical unit of database work comprising one or more `READ` / `WRITE` operations that must obey the **ACID** properties (Atomicity, Consistency, Isolation, Durability).
- **What Are Locks?** Synchronization primitives managed by our `LockManager` / `LockRepository` (`backend/app/repositories/lock_repository.py:46-129`) under **Strict Two-Phase Locking (Strict 2PL)**:
  - **Shared Lock (`S`):** Acquired before a `READ` operation. Multiple transactions can hold `S` locks on the same `data_item` simultaneously.
  - **Exclusive Lock (`X`):** Acquired before a `WRITE` operation. Incompatible with both `S` and `X` locks held by any other transaction (`conflict = (req == X and len(other_locks) > 0) or (req == S and any(L.type == X for L in other_locks))`). Also supports in-place **Lock Upgrades (`S → X`)** if no other transaction holds `S`.
- **What Is a Deadlock & How Does It Occur?** A permanent blocking state where a set of two or more transactions are each waiting for a lock held by another transaction in the set (e.g., T_1 holds `X(Row_A)` and requests `X(Row_B)`, while T_2 holds `X(Row_B)` and requests `X(Row_A)`). It occurs when all four of **Coffman's Conditions** hold simultaneously:
  1. **Mutual Exclusion:** Exclusive (`X`) locks cannot be shared.
  2. **Hold and Wait:** Under Strict 2PL, a transaction holds existing locks while waiting to acquire new ones.
  3. **No Preemption:** Locks cannot be forcibly taken away without aborting/rolling back the holding transaction.
  4. **Circular Wait:** A closed cycle $T_1 \rightarrow T_2 \rightarrow \dots \rightarrow T_k \rightarrow T_1$ exists in the Wait-For Graph.
- **What Is a Wait-For Graph (WFG) & How OS-DBX Detects Cycles (`backend/app/services/deadlock_service.py:29-64`):**
  - **Vertices:** Active Transactions (T_i).
  - **Directed Edges ($T_{waiter} \rightarrow T_{holder}$):** Added whenever T_{waiter} has a lock with `status='WAITING'` on `data_item` D, T_{holder} has a lock with `status='HELD'` on D ($T_{waiter} \neq T_{holder}$), and at least one of the locks is Exclusive (`X`).
  - **Cycle Detection Algorithm:** `DeadlockService.detect()` runs a recursive **Depth-First Search (DFS)** from every vertex and normalizes discovered cycles using `_canonical_cycle()` (lexicographically smallest cyclic rotation) to eliminate duplicate cycle permutations. When detected, `DeadlockRepository.record_deadlock()` (`deadlock_repository.py:28-186`) links the cycle's transactions to their OS `PID`s and `cross_layer_traces` and inserts a `HIGH`-severity `DEADLOCK` record into `incidents` with full JSON `evidence`!

9.2 Deep Dive: Incident Blast Radius / Causal Impact Analysis (Sub-Feature [3])

- **Where Written:** `BlastRadiusService.analyze_deadlock_incident()` in `backend/app/services/blast_radius_service.py:28-231`, exposed at `GET /api/v1/incidents/{incident_id}/blast-radius` (`routes_blast_radius.py`) and CLI function `run_blast_radius_analysis()` (`cli.py:1423-1460`).
- **How It Works (3-Tier Cascading Impact Rules):**

Impact Level	Affected Resource Types	Rule Applied & Deterministic Logic in `blast_radius_service.py`
DIRECT	`TRANSACTION`, `LOCK`, `OS_PROCESS`	• `DEADLOCK_CYCLE_MEMBER` / `INCIDENT_EVIDENCE_CYCLE`: Every transaction T_i directly inside the deadlock cycle. • `DEADLOCK_CYCLE_RESOURCE` / `DEADLOCK_CONTENDED_ITEM`: Every locked `data_item` contended by cycle transactions. • `TRANSACTION_OWNER_PROCESS`: Every host OS Process (`PID`) owning a deadlocked transaction (`SELECT DISTINCT pid FROM transactions WHERE transaction_id IN (...)`, lines 185–200).
INDIRECT	`TRANSACTION`	• `BLOCKED_BY_CYCLE` / `INCIDENT_WAITS_FOR_GRAPH`: Transactions in the Wait-For Graph that are *not* inside the cycle itself, but are queued waiting behind a deadlocked transaction (lock convoy cascade, lines 100–115 & 165–180).
POTENTIAL	`TRANSACTION`	• `ACTIVE_DURING_INCIDENT`: Other currently `ACTIVE` transactions running concurrently while the incident remains unresolved (lines 117–135).

9.3 Deep Dive: Forensic Execution Replay & HTTP 409 Conflict Guard (Sub-Feature [4])

- **Where Written:** `InvestigationRepository.investigate()` and `save_replay()` in `backend/app/repositories/investigation_repository.py:28-109`, exposed at `GET /api/incidents/{id}/investigation` and `POST /api/incidents/{id}/replay` (`routes_investigation.py:24-50`).
- **What Forensic Execution Replay Is:** Instead of re-running destructive live SQL queries (which could mutate production data), Forensic Replay deterministically reconstructs the exact chronological sequence of events across layers:
  1. `TRACE_STARTED` (from `cross_layer_traces`)
  2. `CORRELATED` OS & Query events (joined across `event_correlations`, `os_events`, and `query_executions`)
  3. `BEGIN` / `READ` / `WRITE` / `COMMIT` / `ROLLBACK` (from `transaction_operations`)
  4. `LOCK_HELD` / `LOCK_WAITING` / `LOCK_RELEASED` (from `locks`)

  It sorts all steps by `(timestamp, sequence)`, re-indexes them `1..N`, and persists an immutable JSON snapshot into `incident_replays` (`outcome='REPLAYED'`).
- **Why the HTTP 409 Conflict Guard Exists (Phase-II Report Challenge #5):** Early in development, replaying an incident that had no linked `trace_id` or no correlated events produced an empty timeline. We hardened `save_replay()` (lines 65–70) to raise `ReplayUnavailableError` (mapped to **HTTP 409 Conflict** in `routes_investigation.py:45-49`) if `investigation.trace is None` or if no `step.source == "correlation"` exists!

9.4 All 5 Sub-Features in CLI Module [5] & Schema Tables

- **CLI Sub-Features:** `[1]` View Recent Deadlock & Incident Records (`GET /api/deadlocks/incidents`), `[2]` Deep-Dive Root Cause Timeline Investigation (`GET /api/incidents/{id}/investigation`), `[3]` Incident Blast Radius & Causality Impact Analysis (`GET /api/v1/incidents/{id}/blast-radius`), `[4]` Forensic Incident Replay (`POST /api/incidents/{id}/replay`), `[5]` Scan Live Active Locks for Deadlocks (`POST /api/deadlocks/detect/live`).
- **Schema Tables:** `incidents` (lines 381–394: includes `severity` and `evidence JSON`), `incident_replays` (lines 431–442), `locks` (lines 285–297), `transactions` (lines 236–247), `transaction_operations` (lines 250–261), `cross_layer_traces` (lines 338–354), and `event_correlations` (lines 357–378).

SECTION 10 • CLI MODULE [6]: PERFORMANCE & BENCHMARK WORKLOAD LAB

10. What Is the "Performance & Benchmark Workload Lab", Is It Really Live, How Do We Test It Step-by-Step, and What Is Its Codebase, Schema & Concept Foundation?

30-SECOND VIVA PITCH (DIRECT ANSWER FOR EXAMINER)

Yes — the Performance & Benchmark Workload Lab (Main Menu Option `[6]`) executes real, live database transactions and lock contention workloads against MySQL via `BenchmarkRunnerService` (`backend/app/services/benchmark_service.py`) while simultaneously measuring wall-clock duration with `time.perf_counter()` and sampling live host CPU% and RSS memory overhead via `psutil`. In addition, the OS Monitor package contains `OSBenchmarkEngine` (`os_monitor/benchmark/engine.py` & `scenarios.py`) with 6 live OS stress generators (file creation, read/write, rename/delete, directory ops, process spawning, and multiprocessing CPU stress).

10.1 Why This Feature Is Needed & How It Works in Code

- **Why It Is Needed:** Any observability platform must answer two performance questions: (1) *How does the database perform under pure transaction load vs. hot-row lock contention?* and (2) *How much CPU, RAM, and latency overhead does the observability platform itself add?*
- **Codebase Files:**
  - `cli.py` : `performance_menu()` (lines 1631–1715) and `run_benchmark()` (lines 1595–1630).
  - `backend/app/api/routes_benchmark.py` (lines 1–38): `GET /api/v1/benchmarks/scenarios` and `POST /api/v1/benchmarks/run` .
  - `backend/app/api/routes_performance.py` (lines 1–261): `POST/GET /api/performance/records` and `GET /api/performance/system-health` .
  - `backend/app/services/benchmark_service.py` (lines 23–164): `BenchmarkRunnerService` .
  - `os_monitor/benchmark/engine.py` & `os_monitor/benchmark/scenarios.py` : `OSBenchmarkEngine` + 6 live OS workload scenarios (`FileCreateScenario` , `FileOpenReadWriteScenario` , `FileRenameDeleteScenario` , `DirectoryOperationsScenario` , `ProcessOperationsScenario` , `ResourceStressScenario`).
- **How the 3 Live Benchmark Scenarios Work in `BenchmarkRunnerService` (`benchmark_service.py:131-163`):**
  1. **`DBMS_ONLY` (`_run_dbms_only` , lines 131–142):** Loops `operations` times, executing real MySQL `INSERT` s into `processes` , `transactions` , `transaction_operations` , and `system_calls` via `transaction_service.begin_transaction()` , followed by `UPDATE transactions SET status='COMMITTED'` via `commit_transaction()` .
  2. **`LOCK_CONTENTION` (`_run_lock_contention` , lines 144–157):** Loops `operations` times, starting a real transaction, acquiring an **Exclusive (`X`) lock** on a single shared hot resource (`data_item = "benchmark_hot_row"`) in the `locks` table using `SELECT ... FOR UPDATE` , and committing/releasing the lock.
  3. **`DEADLOCK` (`_run_deadlock` , lines 159–163):** Executes deterministic transaction cycles without hanging synchronous HTTP workers.
- **Automatic Telemetry Persistence (lines 97–119):** Every run automatically writes two rows into `performance_records` with `data_source = 'BENCHMARK'` : `benchmark_<scenario>_duration` (in ms) and `correlation_latency_ms` .

10.2 Step-by-Step Live Testing Procedure for the Examiner

1. **Step 1:** In the running CLI (`python cli.py`), select Option `[6] Performance & Benchmark Workload Lab` .
2. **Step 2 (Run Live Benchmark — Sub-Option `[2]`):**
   - Press `2` (`Run Live Benchmark Workload Simulator`).
   - When prompted for scenario `[DBMS_ONLY/LOCK_CONTENTION/DEADLOCK]` , type `LOCK_CONTENTION` and press Enter.
   - When prompted `Enter operation count:` , type `25` and press Enter.
   - Observe the `Benchmark Results [LIVE]` table showing `Scenario: LOCK_CONTENTION` , real wall-clock `Total Time (ms)` , `Ops Executed: 25` , `Failed Ops: 0` , and `OS Dropped Events: 0` .
3. **Step 3 (Compare Against Pure Transaction Load):** Press `2` again, select `DBMS_ONLY` with `25` operations, and show the examiner how `LOCK_CONTENTION` takes longer due to extra Strict 2PL row-lock acquisition queries on `benchmark_hot_row` !
4. **Step 4 (Verify Persistence in MySQL — Sub-Option `[1]`):** Press `1` (`View Recent Performance Records`). Point out the newly created rows at the top of the table with `Source = BENCHMARK` alongside `LIVE_MYSQL` and `SIMULATION` metrics.
5. **Step 5 (Record Custom Metric — Sub-Option `[3]`):** Press `3` , enter metric name `viva_demo_latency_ms` , value `9.45` , press Enter to skip `trace_id` , and press `1` again to show it persisted in MySQL.

10.3 Schema Tables & OS / DBMS / CN Concepts Used

- **Schema Tables:** `performance_records` (`record_id` PK, `trace_id` FK, `metric_name` , `metric_value` , `data_source` , `recorded_at`), `query_executions` , `dbms_query_observations` , and `resource_metrics` .
- **OS CONCEPTS** High-resolution monotonic hardware timers (`time.perf_counter()`), Resident Set Size (RSS) virtual memory accounting, and bounded producer-consumer backpressure metrics (`dropped_events`).
- **DBMS CONCEPTS** Hot-row lock contention (`SELECT ... FOR UPDATE` on `benchmark_hot_row`), transaction commit throughput (TPS), and observability overhead isolation.
- **CN CONCEPTS** Synchronous HTTP REST payload serialization latency and pooled TCP connection reuse (`MySQLConnectionPool(pool_size=10)`).

SECTION 11 • CLI MODULE [7]: OS SIMULATION LAB (SCHEDULING, MEMORY & DEADLOCK THEORY)

11. How Does the OS Simulation Lab Work? Explain FCFS, SJF, SRTF, RR, Priority, Gantt Charts, Exact Formulas, FIFO/LRU/Optimal Page Replacement, Banker's Algorithm, RAG, Codebase & Schema Tables.

30-SECOND VIVA PITCH (DIRECT ANSWER FOR EXAMINER)

The **OS Simulation Lab** (Main Menu Option `[7]`, `os_simulation_menu()` in `cli.py:1884-1921`, backed by `os_monitor/simulations/` and `backend/app/api/routes_simulations.py`) provides a deterministic algorithmic sandbox for three core Operating System domains: (1) **CPU Scheduling** (`FCFS`, `SJF`, `SRTF`, `Round Robin`, `Priority`) with Gantt chart generation, (2) **Virtual Memory Page Replacement** (`FIFO`, `LRU`, `Optimal` + `ALL` side-by-side comparison), and (3) **OS Deadlock Avoidance & Detection** (`Banker's Safety Algorithm` & `Resource Allocation Graph` cycle detection), persisting every experiment into 7 dedicated SQL tables.

11.1 Why We Developed the OS Simulation Lab in OS-DBX

1. **Controlled Baseline vs. Noisy Live Host:** Live OS telemetry is non-deterministic because hundreds of background OS daemons run simultaneously. The Simulation Lab allows students and engineers to run controlled, repeatable experiments and compare theoretical scheduling/paging/deadlock behavior against live system metrics.
2. **Full Persistence & Historical Auditing:** Every simulation run is stored in MySQL (`simulations`, `scheduled_processes`, `memory_experiments`, `page_references`, `deadlock_experiments`, and `performance_records`).

11.2 Sub-Feature [1]: CPU Scheduling Simulator & Exact Mathematical Formulas

Implemented in `os_monitor/simulations/scheduling/` and exposed at `POST /api/simulations/scheduling` (`routes_simulations.py:78-174`):

Algorithm	File & Function	Preemptive?	Exact Selection & Dispatch Logic in Code
FCFS (First-Come, First-Served)	`fcfs.py:36-68` (`fcfs()`)	Non-Preemptive	Sorts processes by `arrival_time` (`sorted(processes, key=lambda p: p.arrival_time)`). If `current_time < p.arrival_time`, advances clock. Runs each process to completion. Susceptible to the **Convoy Effect**.
SJF (Shortest Job First)	`sjf.py:9-53` (`sjf()`)	Non-Preemptive	Filters arrived processes (`p.arrival_time ≤ current_time`) and dispatches `min(ready, key=lambda x: (x.burst_time, x.arrival_time))`. Provably optimal for minimum average waiting time among non-preemptive schedulers.
SRTF (Shortest Remaining Time First)	`srtf.py:10-65` (`srtf()`)	**Preemptive** (1-tick granularity)	Steps 1 time unit per iteration. At every tick, selects `min(ready, key=lambda x: (x.remaining_time, x.arrival_time))`. If a newly arrived process has a shorter remaining time than the running process, it preempts the CPU immediately.
Round Robin (RR)	`round_robin.py:11-79` (`round_robin()`)	**Preemptive** (Time Quantum q)	Maintains a FIFO `collections.deque`. Runs front process for `run_time = min(quantum, p.remaining_time)`. Enqueues newly arrived processes *before* re-appending the preempted process if `p.remaining_time > 0`.
Priority Scheduling	`priority.py:10-52` (`priority_scheduling()`)	Non-Preemptive	Lower priority number = higher urgency (UNIX `nice` convention). Dispatches `min(ready, key=lambda x: (x.priority, x.arrival_time))`.

Gantt Chart Generation & Exact Scheduling Formulas:

- **Gantt Chart Construction:** Each scheduler builds `gantt = [{"pid": p.pid, "start": t1, "end": t2}, ...]`. In `srtf.py:39-42`, adjacent 1-tick slices belonging to the same `pid` are automatically merged (`if gantt and gantt[-1]["pid"] == p.pid: gantt[-1]["end"] = current_time + 1`). Rendered in CLI (`cli.py:1769`) as `P1 (0-6) | P2 (6-10) | P3 (10-12)`.
- **Exact Mathematical Formulas Computed in Every Scheduler:**

  1. Turnaround Time $(TAT_i) = CT_i - AT_i$ (`p.turnaround_time = p.finish_time - p.arrival_time`)
  2. Waiting Time $(WT_i) = TAT_i - BT_i$ (`p.waiting_time = p.turnaround_time - p.burst_time`)
  3. Response Time $(RT_i) = First_CPU_Start_Time_i - AT_i$ (`p.response_time = p.start_time - p.arrival_time`)
  4. Average WT $= (\Sigma\ WT_i)\ /\ N$ • Average TAT $= (\Sigma\ TAT_i)\ /\ N$ • CPU Utilization (%) $= [(\Sigma\ BT_i)\ /\ Total_Time] \times 100$ • Throughput $= N\ /\ Total_Time$

11.3 Sub-Feature [2]: Memory Page Replacement Lab (FIFO, LRU, Optimal)

Implemented in `os_monitor/simulations/memory/` and exposed at `POST /api/simulations/memory` (`routes_simulations.py:177-281`):

- **FIFO (First-In, First-Out — `fifo.py:21-59`):** Maintains an insertion-order queue `order`. On a page fault when `len(frames) == num_frames`, evicts `oldest = order.pop(0)`. Susceptible to **Belady's Anomaly** (increasing frames can sometimes increase page faults).
- **LRU (Least Recently Used — `lru.py:9-50`):** Tracks `last_used[page] = index` on every reference (hit or fault). On a fault when full, evicts `lru_page = min(frames, key=lambda p: last_used.get(p, -1))`. Stack algorithm (immune to Belady's Anomaly).
- **Optimal / Belady's Algorithm (`optimal.py:10-51`):** Looks ahead in the future reference string `range(i + 1, n)` and evicts the resident page whose next reference is furthest in the future (`float('inf')` if never used again: `victim = max(frames, key=next_use)`). Provides the theoretical lower bound on page faults.

- `ALL` **Comparison Mode (** `routes_simulations.py:223-235` **):** Runs FIFO, LRU, and Optimal simultaneously on the same reference string and frame count for side-by-side comparison.
- **Exact Memory Formulas (*N* = length of reference string):**

  $$\text{Page Hits (H)} = \text{N} - \text{Page_Faults (F)} \quad \bullet \quad \text{Fault Rate} = \text{F} / \text{N} \quad \bullet \quad \text{Hit Ratio (Hit Rate)} = \text{H} / \text{N} = 1 - \text{Fault_Rate}$$

11.4 Sub-Feature [3]: OS Deadlock Lab (Banker's Algorithm & Resource Allocation Graph)

- **Banker's Algorithm (** `os_monitor/simulations/deadlock/bankers.py:18-86` **,** `POST /api/simulations/deadlock/bankers` **):**
  - Computes the Remaining Need Matrix: `Need[i][j] = Max[i][j] - Allocation[i][j]`.
  - Validates resource requests (`Request[j] ≤ Need[pid][j]` and `Request[j] ≤ Available[j]`).
  - Runs Dijkstra's Safety Algorithm: initializes `Work = Available[:]`, finds an unfinished process *i* with `Need[i] ≤ Work`, adds its allocation back (`Work += Allocation[i]`), and appends *i* to `safe_sequence` (default preset yields safe sequence `[1, 3, 4, 0, 2]`).
- **Resource Allocation Graph — RAG (** `os_monitor/simulations/deadlock/resource_graph.py:24-104` **,** `POST /api/simulations/deadlock/rag` **):**
  - Builds a directed bipartite graph with Request Edges ($P_i \rightarrow R_j$) and Assignment Edges ($R_j \rightarrow P_i$), running DFS with a recursion stack (`rec_stack`) to detect cycles (e.g., `['P0', 'R1', 'P1', 'R0', 'P0']`).

11.5 Schema Tables for OS Simulation Lab (7 Tables in `database/schema.sql:140-233`)

`simulations` (master record), `scheduled_processes` (per-process *AT, BT, WT, TAT, CT*), `memory_experiments`, `page_references` (per-step fault boolean), `disk_requests`, `deadlock_experiments`, and `synchronization_experiments` (plus summary metrics in `performance_records`). *(In the Phase-II ER diagram, these map to* `simulations`, `scheduled_processes`, `simulation_results`, *and* `simulation_metrics`.*)*

SECTION 12 • CLI MODULE [8]: DBMS SIMULATION LAB (SERIALIZABILITY, WAL RECOVERY & STRICT 2PL)

12. How Does the DBMS Simulation Lab Work? Explain ACID Properties, Conflict Serializability & Precedence Graph Cycle Absence, WAL Crash Recovery (UNDO/REDO), Strict 2PL Locking, Codebase & Schema.

30-SECOND VIVA PITCH (DIRECT ANSWER FOR EXAMINER)

The **DBMS Simulation Lab** (Main Menu Option `[8]`, `dbms_simulation_menu()` in `cli.py:2193-2230`) demonstrates how a relational database engine guarantees **ACID properties** under concurrent execution and system crashes. It implements three core DBMS internals backed by real MySQL persistence: (1) **Conflict Serializability & Precedence Graph Analysis** (`ScheduleService.analyze()`), (2) **ARIES-inspired WAL (Write-Ahead Logging) UNDO/REDO Crash Recovery** (`RecoveryService.recover()`), and (3) **Strict Two-Phase Locking (Strict 2PL)** with Shared (`S`) and Exclusive (`X`) lock compatibility and live deadlock cycle detection.

12.1 ACID Properties & How Each Is Implemented in OS-DBX

ACID Property	Theoretical Definition	Exact Implementation in OS-DBX Codebase
Atomicity ("All or Nothing")	Either all operations of a transaction complete, or none take effect.	`TransactionService.rollback()` (`transaction_service.py:51-53`) & `RecoveryService.recover()` (`recovery_service.py:34-37` `UNDO` phase restores `old_value` for all uncommitted loser transactions).
Consistency	Transactions transition the database from one valid state satisfying all integrity constraints to another.	Enforced via 27+ Foreign Key constraints, `UNIQUE` indexes, and `TransactionStateError` (HTTP 409) preventing operations on already committed/aborted transactions (`transaction_repository.py:63-112`).
Isolation	Concurrent transactions execute as if they were running serially without interfering.	Enforced via **Strict 2PL** (`LockManager` / `LockRepository` acquiring `S` / `X` locks before `READ` / `WRITE`), MySQL isolation levels (`READ_UNCOMMITTED`, `READ_COMMITTED`, `REPEATABLE_READ`, `SERIALIZABLE`), and **Conflict Serializability Precedence Graph** analysis (`ScheduleService`).
Durability	Once a transaction commits, its changes survive power loss or system crashes.	Enforced via `recovery_logs` (Write-Ahead Log), `checkpoints`, and the `REDO` phase in `RecoveryService.recover()` (`recovery_service.py:28-31`) which replays all logged updates forward.

12.2 Sub-Feature [1]: Conflict Serializability & Precedence Graph Analysis (`schedule_service.py`)

- **Where Written:** `ScheduleService.analyze()` in `backend/app/services/schedule_service.py:13-45`, exposed at `POST /api/schedules/analyze` (`routes_schedules.py:14-52`) and CLI function `run_serializability_lab()` (`cli.py:1927-2002`).
- **What Is a Conflict?** Two operations O_i (at step i) and O_j (at step j, where $i < j$) conflict if and only if:
  1. They belong to **different transactions** ($T_a \neq T_b$).
  2. They access the **same data item** (X).
  3. **At least one of them is a Write (`W`)** — producing three conflict pairs: **Read-Write (RW)** (Unrepeatable Read), **Write-Read (WR)** (Dirty Read), and **Write-Write (WW)** (Lost Update). *(Read-Read pairs commute and never conflict.)*
- **Precedence (Serialization) Graph & Why Cycle Absence Proves Conflict Serializability:**
  - For every conflicting pair (O_i, O_j) with $i < j$, `ScheduleService.analyze()` adds a directed edge $T_i \rightarrow T_j$ (`graph[left.transaction_id].add(right.transaction_id)`), meaning T_i must precede T_j in any conflict-equivalent serial schedule.
  - **Mathematical Theorem (Cycle Absence):** A directed graph has a valid **topological sort** (a linear serial ordering $T_{\pi(1)}, T_{\pi(2)}, \ldots, T_{\pi(k)}$ that respects all dependency edges) **if and only if it is a Directed Acyclic Graph (DAG) — i.e., it has zero cycles!** If a cycle $T_1 \rightarrow T_2 \rightarrow T_1$ exists, T_1 must execute before T_2 and T_2 must execute before T_1, which is impossible in any serial schedule.
  - `ScheduleService.analyze()` runs DFS cycle detection with canonical rotation deduplication (`_canonical_cycle()`) and sets `conflict_serializable = not ordered_cycles`.

12.3 Sub-Feature [2]: WAL (Write-Ahead Logging) Crash Recovery (`recovery_service.py`)

- **Where Written:** `RecoveryService.recover()` in `backend/app/services/recovery_service.py:6-45`, exposed at `POST /api/recovery/recover` (`routes_recovery.py:14-53`) and CLI function `run_wal_recovery_lab()` (`cli.py:2004-2101`).
- **Exact 3-Step Recovery Algorithm:**
  1. **Analysis / Classification (lines 14–25):** Sorts WAL records by `sequence` ascending. Identifies **Winner Transactions** (`committed = {e.tx_id | e.log_type == 'COMMIT'}`) and **Loser Transactions** (`losers = all_updating_txs - committed`).
  2. `REDO` **Phase — Repeating History Forward (lines 28–31):** Scans forward in ascending sequence order and applies `state[entry.data_item] = entry.new_value` for all logged `WRITE` / `UPDATE` records (ensuring durability of committed changes).
  3. `UNDO` **Phase — Rolling Back Losers in Reverse Order (lines 34–37):** Scans `reversed(updates)` backwards from the crash point; for every record where `entry.transaction_id in losers`, restores the before-image: `state[entry.data_item] = entry.old_value` (ensuring atomicity). Also records a snapshot in `checkpoints`.

12.4 Sub-Feature [3]: Strict 2PL Lock & Deadlock Lab (`run_2pl_lock_lab()` , `cli.py:2103-2191`)

- Executes a live 6-step Strict 2PL walkthrough against the backend (`POST /api/transactions/begin` , `POST /api/locks` , `POST /api/deadlocks/detect/live` , `POST /api/transactions/{id}/rollback`):
  1. Starts two live transactions T_1 and T_2.
  2. T_1 acquires Exclusive lock `X(Row_Alpha)` → `HELD` .
  3. T_2 acquires Exclusive lock `X(Row_Beta)` → `HELD` .
  4. T_1 requests `X(Row_Beta)` (held by T_2) → enters `WAITING` ($T_1 \rightarrow T_2$).
  5. T_2 requests `X(Row_Alpha)` (held by T_1) → enters `WAITING` ($T_2 \rightarrow T_1$).
  6. Triggers live Wait-For Graph scan (`POST /api/deadlocks/detect/live`), detects cycle $T_1 \rightarrow T_2 \rightarrow T_1$, persists a new `DEADLOCK` incident in `incidents` , and rolls back T_1 and T_2 to release all locks cleanly!

12.5 Schema Tables for DBMS Simulation Lab

`transactions` , `transaction_operations` , `locks` , `schedules` , `schedule_operations` , `recovery_logs` (called `wal_records` in the Phase-II ER diagram), and `checkpoints` .

SECTION 13 • CLI MODULE [9]: WHAT-IF SCENARIO ANALYSIS LAB

13. What Is the "What-If Scenario Analysis" Lab, Why Is It Needed, and How Does It Work Across Its Codebase, Mathematical Models & Schema Tables?

30-SECOND VIVA PITCH (DIRECT ANSWER FOR EXAMINER)

The **What-If Scenario Analysis Lab** (Main Menu Option `[9]`, `what_if_menu()` in `cli.py:2420-2475`, backed by `backend/app/api/routes_what_if.py` and `WhatIfRepository`) is a **counterfactual predictive modeling engine**. It allows system architects and DBAs to ask "*What will happen if we double concurrent transactions, change a schedule's interleaving, crash before a commit, or run 20 processes on 4 CPU cores?*" and get exact mathematical risk assessments and state predictions persisted in `what_if_scenarios` — **without** polluting live production monitoring tables or risking system downtime.

13.1 Why This Feature Is Needed in OS-DBX

- Live monitoring (`Modules 1-5`) is **reactive** (it observes what already happened on the host or in MySQL).
- The What-If Lab makes OS-DBX **proactive and predictive**: engineers can test hypothetical transaction schedules, crash points, CPU core counts, RAM allocations, disk head requests, and lock concurrency levels in isolation before deploying changes.

13.2 Codebase Files & Database Schema Table

- **CLI Functions (`cli.py`):** `what_if_menu()` (lines 2420–2475), `run_whatif_schedule()` (lines 2236–2290), `run_whatif_recovery()` (lines 2292–2318), `run_whatif_os_experiment()` (lines 2368–2419), `list_whatif_scenarios()` (lines 2320–2337), `view_whatif_scenario()` (lines 2339–2366).
- **Backend Files:** `backend/app/api/routes_what_if.py` (lines 1–189), `backend/app/models/what_if.py` (lines 1–44), `backend/app/repositories/what_if_repository.py` (lines 1–98).
- **Schema Table (`database/schema.sql:445-452`):**

```
CREATE TABLE what_if_scenarios (
    scenario_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    analysis_type VARCHAR(50) NOT NULL,  -- 'SCHEDULE', 'RECOVERY', or 'OS_CPU_LOAD' / 'OS_MEMORY_PRESSURE' / etc.
    name VARCHAR(255) NOT NULL,
    input_json LONGTEXT NOT NULL,
    result_json LONGTEXT NOT NULL,
    created_at DATETIME NOT NULL
);
```

13.3 All 5 Sub-Features & Exact Mathematical Models in `routes_what_if.py`

1. **Sub-Feature `[1]`: What-If Schedule Analysis (`POST /api/what-if/schedules`, lines 29–40):** Accepts a named hypothetical operation sequence (`WhatIfScheduleRequest`), runs `ScheduleService().analyze()` to build the precedence graph and check for cycles, and saves the scenario (`analysis_type="SCHEDULE"`).
2. **Sub-Feature `[2]`: What-If Crash Recovery Experiment (`POST /api/what-if/recovery`, lines 43–54):** Accepts a hypothetical `initial_state` and WAL log sequence (`WhatIfRecoveryRequest`), runs `RecoveryService().recover()` to compute the post-crash `REDO` / `UNDO` state, and saves the scenario (`analysis_type="RECOVERY"`).
3. **Sub-Feature `[3]`: What-If OS Workload & Contention Experiments (`POST /api/what-if/os`, lines 57–163):** Evaluates 4 mathematical OS/DBMS bottleneck models:

What-If OS Model	Input Parameters	Exact Equations & Risk Thresholds in `routes_what_if.py`
1. `CPU_LOAD` (lines 66–83)	`process_count` (P), `avg_burst_ms` (B), `core_count` (C)	`Utilization (%) = min(100, [(P × B) / (C × 1000)] × 100)`, `Avg Wait (ms) = max(0, (P − C) × (B / C))` `Throughput = (1000 / B) × C`. **Bottleneck Risk:** `HIGH` (>85%), `MODERATE` (>60%), `LOW` (≤60%).
2. `MEMORY_PRESSURE` (lines 84–114)	`allocated_mb` (A), `total_ram_mb` (T), `frames` (F), `page_references`	`Memory Usage (%) = (A / T) × 100`. Simulates FIFO paging across `page_references` to compute `Fault Rate = faults / N`. **Thrashing Risk:** `CRITICAL` if Usage > 90% or Fault Rate > 0.6, else `STABLE`.
3. `DISK_IO` (lines 115–142)	`current_head` (H_0), cylinder `requests`	Executes greedy **Shortest Seek Time First (SSTF)**: `nearest = argmin |r − cur|`, summing `total_seek` cylinders and `avg_seek_time_ms = total_seek × 0.12`. **I/O Saturation Risk:** `HIGH` if `total_seek > 500`, else `LOW`.
4. `RESOURCE_CONTENTION` (lines 143–156)	`concurrent_tasks` (T_c), `shared_locks` (L_s)	`Contention Index = min(1.0, (T_c × L_s) / 20.0)`, `Lock Convoy Probability = Index × 100%`. **Deadlock Risk:** `ELEVATED` if `Index > 0.6`, else `MINIMAL`.

4. **Sub-Features `[4]` & `[5]`: List & Inspect Saved Scenarios (`GET /api/what-if/scenarios` & `GET /api/what-if/scenarios/{id}`):** Retrieves historical What-If experiments and pretty-prints their `input` and `result` JSON payloads.

SECTION 14 • PROJECT MILESTONES & IN-DEPTH INDIVIDUAL TEAM CONTRIBUTIONS

14. What Overall Achievements Have Been Completed, and What Exactly Did Krishna, Aditya, Akanksha, and Akshita Contribute Across OS, DBMS, Backend, CLI, Testing & Frontend?

30-SECOND VIVA PITCH (DIRECT ANSWER FOR EXAMINER)

Team **CodeX4** has delivered a working end-to-end cross-layer observability and simulation platform comprising **62 Git commits across 14 branches**, a **32-table 3NF MySQL relational database**, a **FastAPI backend with 40+ REST endpoints**, a **2,636-line interactive Rich CLI console**, a **2,249-line browser frontend playground**, and **100 passing automated unit & integration tests**. Below is the exact, code-verified breakdown of each team member's engineering contributions.

14.1 Overall Project Achievements & Deliverables Completed

- **Database Architecture:** Designed and normalized a 32-table MySQL schema (`database/schema.sql`) in 3rd Normal Form (3NF) with 27+ foreign keys, composite unique watermarks, and 607 lines of realistic seed data (`database/seed.sql`).
- **Live Cross-Layer Observability:** Built continuous OS (`WindowsAdapter` + `LiveEventStream`) and DBMS (`MySQLQueryCollector` + `performance_schema`) collectors joined in real time by our 3-pronged `IdentityBridgeService` and 8-factor `CorrelationService`.
- **Concurrency, Forensics & Simulations:** Implemented Strict 2PL locking, live Wait-For Graph deadlock cycle detection, 3-tier Blast Radius analysis, deterministic Forensic Incident Replay (with HTTP 409 validation), 9 OS/DBMS simulation algorithms, and 6 counterfactual What-If models.
- **Verification & Quality Assurance:** Passed **100 automated tests** (`pytest`), built an automated GitHub Actions CI pipeline, and validated live end-to-end OS-to-MySQL socket correlation (`verify_end_to_end.py`, `test_e2e_final.py`).

14.2 Detailed Individual Contributions of All 4 Team Members

Team Member	Primary Ownership Domains	Exact Codebase Files, Modules & Engineering Contributions
1. Krishna Nitin Anasane *(Team Lead)* ID: `24021788` **42 Git Commits**	• 32-Table 3NF DBMS Schema & Migrations • DBMS Core Engines (Transactions, Strict 2PL Locks, Deadlocks, Serializability, WAL Recovery) • Identity Bridge & Cross-Layer Correlation Engine • Blast Radius, Forensic Replay & What-If Lab • JWT Auth & 10-Module Rich CLI Integration	• **Database Schema & Normalization:** Authored `database/schema.sql` (all 32 tables across OS, Simulation, DBMS, Correlation, Replay, What-If, and Audit layers), `database/seed.sql`, `docs/relational_schema.md`, `docs/Normalisation.md` (1NF/2NF/3NF proofs), and `docs/database_data_dict.md`. • **DBMS Internals & Collectors:** Built `TransactionService`, `LockManager` / `LockRepository` (Strict 2PL `S` / `X` locks & upgrades), `DeadlockService` (WFG DFS cycle detection), `ScheduleService` (Precedence Graph), `RecoveryService` (WAL UNDO/REDO), and `MySQLQueryCollector` (`mysql_query_collector.py` harvesting `performance_schema` with 3-tier fallback). • **Cross-Layer Core & Forensics:** Engineered `IdentityBridgeService` (`identity_bridge.py`: `netstat -ano` + `psutil` + `processlist` + `session_connect_attrs`), `CorrelationService` & `EvidenceService` (PID recycling guard & 8-factor score), `BlastRadiusService`, `InvestigationRepository` (timeline & replay with HTTP 409 guard), and `BenchmarkRunnerService`. • **Security & CLI Console:** Implemented JWT + `bcrypt` auth (`auth.py`, `routes_auth.py`), connection pooling (`database.py`), lifespan background daemons in `main.py`, and the 2,636-line interactive CLI (`cli.py`).
2. Aditya Kumar ID: `24011059` **14 Git Commits** + 6 PR Merges	• OS Process, CPU, Memory & Disk I/O Monitoring • Watchdog Filesystem Event Collector • `WindowsAdapter` & `LiveEventStream` Queue • OS Simulation Lab (Scheduling, Memory, Banker's, RAG) • OS Benchmark Engine & Frontend Collaboration	• **Live Host OS Collectors:** Built `os_monitor/collector.py` (`snapshot_processes()`, `diff_snapshots()`, `build_event()` polling process spawns/exits & states via `psutil`), `os_monitor/resource_monitor.py` (`collect_metrics()` with CPU 1.0% / RAM 0.5% noise thresholds and PID 0 filtering), and `os_monitor/filesystem_monitor.py` (recursive `watchdog` observer for create/modify/delete/move events). • **OS Streaming & Adapter Architecture:** Engineered `NormalizedOSEvent` (`os_monitor/core/events.py` with `capture_latency_sec` and `5/5` `evidence_completeness`), `LiveEventStream` (`os_monitor/core/stream.py` burst-safe 10,000-event buffer), and `WindowsAdapter` (`windows_adapter.py` with thread enumeration, process tree reconstruction, and octal file permissions). • **OS Simulation Lab & OS Benchmarks:** Implemented all 10 OS simulation modules under `os_monitor/simulations/` (`fcfs.py`, `sjf.py`, `srtf.py`, `round_robin.py`, `priority.py`, `fifo.py`, `lru.py`, `optimal.py`, `bankers.py`, `resource_graph.py`), `os_monitor/benchmark/engine.py` & `scenarios.py`, and collaborated on frontend/OS visualization design.

3. Akanksha Balonie ID: `24022979` **Backend, CLI Testing & DevOps Track**	• FastAPI Backend & REST API Integration • DBMS & Backend CLI End-to-End Testing • CI/CD Automation Pipeline • Runtime System Diagnostics & Documentation	• **Backend & CLI Integration Testing:** Collaborated with Krishna on wiring FastAPI REST routers (`backend/app/api/routes_*.py`), validating Pydantic request/response contracts, and testing all 10 interactive CLI modules against live MySQL and in-memory storage modes (`test_e2e_integration.py`, `test_e2e_final.py`, 100 passing automated tests). • **Automated CI Pipeline (Commit `767fec2`):** Created `.github/workflows/ci.yml` running automated backend and OS collector test suites on push/PR across branches. • **Environment Diagnostics & Contributor Standards:** Built `tools/system_diagnostics.py` (202 lines) to automatically audit Python environment readiness, required packages, OS collector permissions, MySQL connectivity, and Performance Schema consumer status prior to live demos, plus authored `CONTRIBUTING.md`.
4. Akshita Negi ID: `24022885` **5 Git Commits** (`origin/front-end` & patch branches)	• Web Frontend Application & Dashboard UI • Browser Process Scheduling Visualizer • Live Backend Health Status Integration • Documentation & Collector Refinements	• **Web Frontend Implementation (Commit `b03b39e` on `origin/front-end` — 2,249 lines):** Built `index.html` (531 lines), `styles.css` (904 lines), and `app.js` (766 lines). • **Interactive Features Built:** Designed the 9-Lab sidebar navigation, the "Challenge: Try all four rules" progress tracker, live backend health polling (`GET http://127.0.0.1:8000/health` and `/health/details` displaying real-time Backend Online/Offline status and Event Stream `Seen`/`Handled`/`Missed` counters), and the interactive **Lab 01: Process Lab** with custom numeric steppers, play/pause/step timeline controls, live Queue/CPU/Finished zones, Gantt chart animation, and "Words to know" educational popups. • **Additional Commits:** Authored 4 patch branches (`axitanegi-patch-1`..`4`) refining `os_monitor/collector.py` and `README.md`.

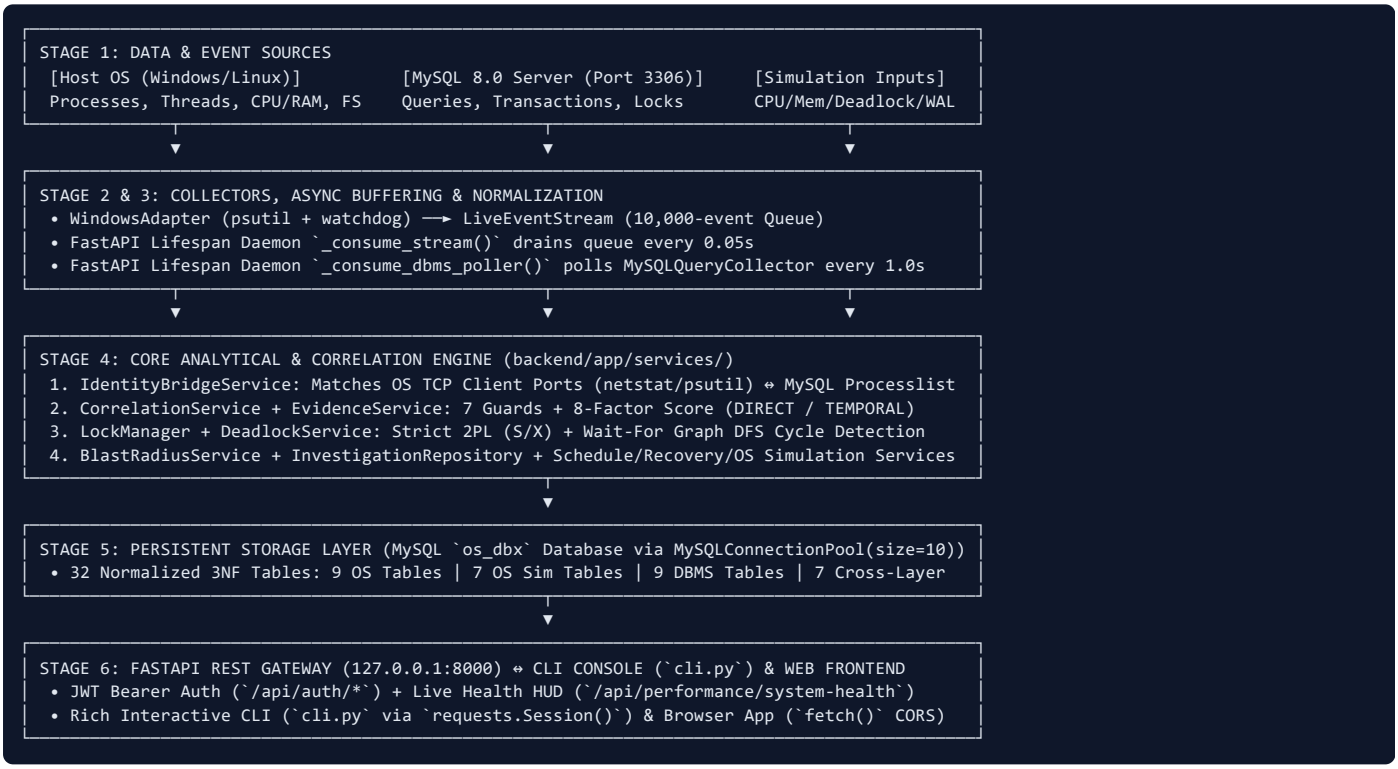
SECTION 15 • COMPLETE SYSTEM ARCHITECTURE, CONCEPT MATRIX & END-TO-END API PIPELINING

15. What Is the Full System, Backend, and Frontend Architecture, What Overall OS, DBMS & CN Concepts Are Used, and How Do All APIs & Workflows Pipeline Together?

30-SECOND VIVA PITCH (DIRECT ANSWER FOR EXAMINER)

OS-DBX follows a **6-Stage Layered Event-Driven & Service-Oriented Architecture**: (1) **Event Sources** (Host OS Kernel, MySQL Server, Simulation Generators) → (2) **Collectors** (`WindowsAdapter`, `MySQLQueryCollector`) → (3) **Async Buffering & Normalization** (`LiveEventStream` 10,000-slot queue, `_consume_stream`, `_consume_dbms_poller`) → (4) **Core Analytical Engine** (`IdentityBridgeService`, `CorrelationService`, `DeadlockService`, `BlastRadiusService`) → (5) **Storage Layer** (`MySQLConnectionPool` + 32 3NF tables in `os_dbx`) → (6) **Presentation & Access Layer** (FastAPI REST Gateway on `127.0.0.1:8000` consumed by our 10-Module Rich CLI and Web Frontend).

15.1 End-to-End 6-Stage System Architecture & Data Flow Pipeline



15.2 Backend Architecture vs. Frontend & CLI Architecture

- **Backend Architecture (4-Tier Clean Architecture in `backend/app/`):**
  1. **Presentation / Router Tier (`backend/app/api/routes_*.py`):** 16 modular FastAPI routers handling HTTP status codes (`200`, `201`, `400`, `401`, `403`, `404`, `409 Conflict`, `422`), JWT dependency injection (`Depends(get_current_user)`), and CORS middleware (`CORSMiddleware(allow_origins=["*"])` in `main.py:386-392`).
  2. **Domain / Service Tier (`backend/app/services/*.py`):** Pure business logic (`IdentityBridgeService`, `CorrelationService`, `EvidenceService`, `TransactionService`, `LockManager`, `DeadlockService`, `BlastRadiusService`, `ScheduleService`, `RecoveryService`, `BenchmarkRunnerService`).
  3. **Data Access / Repository Tier (`backend/app/repositories/*.py`):** Encapsulates parameterized SQL queries, transaction commits/rollbacks, and row locking (`SELECT ... FOR UPDATE`).
  4. **Infrastructure & Lifespan Tier (`backend/app/config/database.py` & `main.py`):** Manages `MySQLConnectionPool(pool_size=10)` and runs the two background daemon tasks (`_consume_stream` and `_consume_dbms_poller`).
- **Frontend & CLI Dual-Client Architecture:**
  1. **Rich Interactive CLI Client (`cli.py`):** Uses a persistent `requests.Session()` with TCP keep-alive, attaches `Authorization: Bearer ` via `get_headers()`, renders the live 4-row telemetry banner on every menu screen, and orchestrates all 10 operational and simulation modules.
  2. **Browser Frontend Client (`index.html`, `styles.css`, `app.js` on `origin/front-end`):** Connects to `http://127.0.0.1:8000/health` and `/health/details` via async `fetch()` calls (enabled by FastAPI's `CORSMiddleware`) to display live backend connectivity and stream counters (`Seen`, `Handled`, `Missed`), while running an interactive browser-based CPU Scheduling engine (`FCFS`, `SJF`, `Priority`, `Round Robin`) with step-by-step timeline playback that works even when offline!

15.3 Master Synthesis Matrix of All OS, DBMS & Computer Networks (CN) Concepts

Domain	Concept Name	What It Defines	Where It Is Used in OS-DBX

OS	**Process Control Block (PCB) & 5-State Lifecycle**	Kernel data structure storing PID, PPID, UID, CPU/RAM usage, and state (`RUNNING`, `SLEEPING`, `STOPPED`, `ZOMBIE`).	`os_monitor/collector.py`, `WindowsAdapter`, and the `processes` & `threads` SQL tables.
OS	**PID Recycling & Lifespan Windowing**	OS kernels reuse PIDs of terminated processes, which can corrupt historical event attribution unless bounded by `[start_time, stop_time]`.	`CorrelationService.correlate()` (`correlation_service.py:114-146`) PID Recycling Guard.
OS	**Producer-Consumer Bounded Buffer**	Thread-safe FIFO queue decoupling fast event producers from slower database consumers with drop-count tracking.	`LiveEventStream` (`queue.Queue(maxsize=10000)` in `os_monitor/core/stream.py`) & `_consume_stream()`.
OS	**CPU Scheduling, Paging & Deadlock Theory**	FCFS, SJF, SRTF, RR, Priority; FIFO, LRU, Optimal paging; Banker's Safety Algorithm & Resource Allocation Graph (RAG).	`os_monitor/simulations/**` & `backend/app/api/routes_simulations.py`.
DBMS	**3rd Normal Form (3NF) & Referential Integrity**	Eliminating transitive and partial functional dependencies using primary/foreign keys and composite unique constraints.	`database/schema.sql` (32 tables, 27+ FKs) & `docs/Normalisation.md`.
DBMS	**ACID & Strict Two-Phase Locking (Strict 2PL)**	Acquiring Shared (`S`) / Exclusive (`X`) locks before Read/Write and holding all locks until `COMMIT`/`ROLLBACK` to prevent cascading aborts.	`TransactionService`, `LockManager`, and `LockRepository` (`lock_repository.py:46-162`).
DBMS	**Wait-For Graph (WFG) & Precedence Graph**	Directed graph cycle detection (DFS): cycle presence in WFG = Deadlock; cycle absence in Precedence Graph = Conflict Serializable.	`DeadlockService.detect()` (`deadlock_service.py`) & `ScheduleService.analyze()` (`schedule_service.py`).
DBMS	**Write-Ahead Logging (WAL) UNDO/REDO Recovery**	Repeating history forward (`REDO`) for durability, then rolling back uncommitted loser transactions in reverse order (`UNDO`) for atomicity.	`RecoveryService.recover()` (`recovery_service.py`), `recovery_logs`, and `checkpoints`.
DBMS	**MySQL Performance Schema Instrumentation**	Non-blocking internal ring buffers (`events_statements_history_long`) recording SQL text, picosecond `TIMER_WAIT`, and `THREAD_ID`.	`MySQLQueryCollector` (`mysql_query_collector.py`) & `dbms_query_observations`.
CN	**TCP 4-Tuple Ephemeral Port & Loopback Socket Matching**	Identifying local IPC endpoints via `(127.0.0.1, ephemeral_port, 127.0.0.1, 3306)` in `ESTABLISHED` TCP state.	`IdentityBridgeService` (`identity_bridge.py:40-115`) via `netstat -ano` and `psutil.net_connections`.
CN	**RESTful HTTP/JSON, CORS & JWT Bearer Auth**	Stateless client-server communication over HTTP/1.1 using `Authorization: Bearer `, status codes (`200`, `401`, `409`), and CORS headers.	`backend/app/main.py`, `backend/app/core/auth.py`, `cli.py`, and `app.js`.

15.4 Complete REST API Surface & Endpoint Catalog

Module / Router	HTTP Endpoints Exposed	Connected Service & Repository
Health & Diagnostics (`main.py`)	`GET /health`, `GET /health/details`	Checks MySQL connectivity, `LiveEventStream` metrics, and `WindowsAdapter` thread status.
Auth & RBAC Audit (`routes_auth.py`, `routes_audit.py`)	`POST /api/auth/register`, `POST /api/auth/token`, `GET /api/auth/me`, `POST /api/audit/log`, `GET /api/audit/`	`auth.py` (`bcrypt`, `python-jose` JWT) → `users` and `audit_logs` tables.
OS Events & Observatory (`routes_events.py`, `routes_os_events.py`)	`POST/GET /api/events`, `POST/GET /api/os-events`, `POST/GET /api/os-events/filesystem`, `GET /api/os-events/processes`, `GET /api/os-events/resources`, `GET /api/os-events/metrics`, `GET /api/os-events/status`, `POST /api/os-events/start`	`EventService`, `EventRepository`, `os_monitor/collector.py`, `resource_monitor.py`, `filesystem_monitor.py`.
DBMS Query Observatory (`routes_dbms_events.py`, `routes_query_executions.py`)	`GET /api/dbms-events/status`, `POST /api/dbms-events/collect`, `GET /api/dbms-events/observations`, `POST /api/query-executions`	`MySQLQueryCollector`, `DBMSObservationRepository`, `QueryRepository`.
Identity Bridge & Correlation (`routes_bridge.py`, `routes_correlation.py`, `routes_demo.py`)	`GET /api/bridge/live-map`, `POST /api/correlations`, `POST /api/correlations/auto`, `GET /api/demo/correlation`	`IdentityBridgeService`, `CorrelationService`, `EvidenceService`, `CorrelationRepository`, `AutoCorrelationRepository`.
Transactions, Locks, Deadlocks & Forensics (`routes_transactions.py`, `routes_locks.py`, `routes_deadlocks.py`, `routes_investigation.py`, `routes_blast_radius.py`)	`POST /api/transactions/begin`, `POST /api/transactions/{id}/read|write|commit|rollback`, `POST /api/locks`, `POST /api/deadlocks/detect`, `POST /api/deadlocks/detect/live`, `GET /api/deadlocks/incidents`, `GET /api/incidents/{id}/investigation`, `POST/GET /api/incidents/{id}/replay(s)`, `GET /api/v1/incidents/{id}/blast-radius`	`TransactionService`, `LockManager`, `DeadlockService`, `BlastRadiusService`, `InvestigationRepository`.

Performance & Benchmarks (`routes_performance.py` , `routes_benchmark.py`)	`GET /api/performance/system-health` , `POST/GET /api/performance/records` , `GET /api/v1/benchmarks/scenarios` , `POST /api/v1/benchmarks/run`	`BenchmarkRunnerService` , `PerformanceRepository` , `psutil` telemetry aggregator.
OS/DBMS Simulations & What-If Lab (`routes_simulations.py` , `routes_schedules.py` , `routes_recovery.py` , `routes_what_if.py`)	`POST /api/simulations/scheduling|memory|deadlock/bankers|deadlock/rag|disk|sync` , `GET /api/simulations/history` , `POST/GET /api/schedules/analyze|history` , `POST/GET /api/recovery/recover|history` , `POST /api/what-if/schedules|recovery|os` , `GET /api/what-if/scenarios/{id}`	`os_monitor/simulations/*` , `ScheduleService` , `RecoveryService` , `WhatIfRepository` .

15.5 Appendix: Reconciling the Phase-II Report ER Diagram (31 Tables) with `database/schema.sql` (32 Tables)

If an examiner compares the conceptual 31-table ER diagram on Page 2 of the Phase-II Report against `database/schema.sql` (which has 32 physical `CREATE TABLE` statements: 28 core + 3 migration tables + 1 `audit_logs` table), use this exact mapping:

ER Diagram Category (Page 2 of Report)	Conceptual Table Names in ER Diagram	Exact Physical SQL Table Names in `database/schema.sql`
1. User & System Management	`users` , `user_sessions` , `system_settings`	`users` (lines 4–11) + `audit_logs` (lines 455–462); sessions are stateless JWTs (`auth.py`) and settings use Pydantic `Settings` (`settings.py`).
2. OS Monitoring & 8. File System	`processes` , `os_events` , `resource_usage` , `filesystem_events` , `network_events` , `directories` , `files`	`processes` , `threads` , `system_calls` , `os_events` (stores process + filesystem events), `resource_metrics` (stores CPU/RAM/Disk/Net I/O), `files` , `directories` , `permissions` .
3. Simulation	`simulations` , `scheduled_processes` , `simulation_results` , `simulation_metrics`	`simulations` , `scheduled_processes` , `memory_experiments` , `page_references` , `disk_requests` , `deadlock_experiments` , `synchronization_experiments` .
4. DBMS Core & 9. Recovery/Logging	`databases` , `transactions` , `queries` , `locks` , `deadlocks` , `wal_records` , `checkpoints`	`transactions` , `transaction_operations` , `query_executions` , `dbms_query_observations` , `locks` , `schedules` , `schedule_operations` , `recovery_logs` , `checkpoints` .
5. Performance, 6. Correlation & 7. Investigation	`performance_records` , `query_statistics` , `resource_usage_logs` , `process_connections` , `identity_bridge` , `event_correlations` , `cross_layer_traces` , `trace_steps` , `incidents` , `incident_events` , `investigations` , `replay_sessions` , `whatif_scenarios`	`performance_records` , `cross_layer_traces` , `event_correlations` , `incidents` (stores deadlock & incident JSON evidence), `incident_replays` , `what_if_scenarios` (while `IdentityBridgeService` & `InvestigationRepository` compute live socket maps and timelines).